DISTRIBUTED MULTI-REASONING MECHANISM TO SUPPORT CONCEPTUAL STRUCTURAL DESIGN

By Lucio Soibelman[1] and Feniosky Peña-Mora,[2] Members, ASCE

ABSTRACT: The conceptual phase of structural design involves selecting preliminary materials, selecting the overall structural form of the building, producing a rough dimensional layout, and considering technological possibilities. Decisions are made on the basis of such information as height of the building, building use, typical live load, wind velocity, earthquake loading, design fundamental period, design acceleration, maximum lateral deflection, spans, story height, and other client requirements. More detailed information about the task itself, constraints, possible solution principles, and known solutions for similar problems is extremely useful in the process of defining and finding a solution to the design problem. This paper presents the M-RAM, which is intended to assist engineers in the conceptual phase of the structural design of tall buildings by providing designers with adapted past design solutions generated by a distributed multi-reasoning mechanism. The objective of the M-RAM (Multi-Reasoning Model) is to provide the adapted past design solutions in an organized and reliable way, creating a support system to enhance creativity, engineering knowledge, and experience of designers. To test the feasibility of the proposed model, a prototype of a distributed artificial intelligence system was developed in which the Internet was used as a communication backbone for the different systems that implemented the reasoning mechanisms.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

INTRODUCTION

The writers of this paper take the view that design is an information-processing activity. After each information output, it might become necessary to improve on the result of the previous step and to repeat each step until the necessary improvements have been achieved. Every design first involves a full understanding of the problem and what information is known about it. The knowledge, ability, and experience of the designers are obviously important factors in developing the initial design, but for any designer, detailed information about the specific task, constraints, possible solution principles, and known solutions for similar problems is necessary to identify the precise nature of the design requirements.

The conceptual design phase is the first step in the design process. After knowing the problem definition, a designer develops an initial design estimate consistent with the designer's experience and knowledge, the constraints, and the requirements. The efficiency and success of the design process depends heavily on this initial estimate. Because this design phase requires knowledge, use of past experience, rules of thumb, intuition, and so forth, it is very difficult to use any procedural programming language for its computerization (Maher 1987).

In the conceptual design phase, past experience plays a vital role, where the process is often assisted by identifying, retrieving, and modifying appropriate past design cases and generalizing the cases, with the implied benefit of being able to use relevant past experience (Manfaat et al. 1997).

This paper presents both the M-RAM (Multi-Reasoning Model) (Soibelman 1998) and a prototype that was developed to evaluate the feasibility of the proposed model. The model and prototype are intended to assist engineers in the initial stages of the structural design of tall buildings by providing designers with adapted past design solutions generated by a distributed multi-reasoning mechanism. The final objective is to help designers to employ past experience and avoid past mistakes. On the basis of functional, architectural, and aesthetic requirements, M-RAM arrives at a few possible configurations for the structural systems, mainly based on geometry and load distribution for gravity and lateral loads, such as dead, live, wind, and earthquake loading.

The conceptual design is a complex problem that cannot be solved by only one reasoning mechanism. When facing this challenge, designers use a combination of heuristic cases, first principles, and past experience to reduce the complexity of the problem and to come up with possible solutions. Instead of building general function modules, the M-RAM model applies an agentlike approach to develop competence modules, each of which is an expert at a particular small task. Each module does all representation, computation, reasoning, and execution related to its particular competence. The emphasis in the proposed architecture is on a more direct coupling of perception to action, distribution, and decentralization, dynamic interaction with the environment, and intrinsic mechanisms to cope with limitations in resources and incomplete knowledge. The traditional positive aspects of distributed processing systems, such as performance, reliability, modularity, and resource-sharing, motivate the development of this architecture (Maes 1991). The application of this approach allows us to overcome the difficulties presented by traditional procedural programming languages to automate or support the conceptual phase of structural design for tall buildings.

Although many different machine-learning algorithms and reasoning mechanisms have been applied before to support the development of conceptual designs, the user had to mediate those programs by hand (Maher and Fenves 1985; Maher 1987; Fenves et al. 1995; Maher and de Silva Garza 1996; Manfaat et al. 1997). With M-RAM, research focused on examining the interconnection and distribution of existing algorithms, mechanisms, and models incorporating existing application programs through a computer network, taking advantage of the Internet's large potential.

CONCEPTUAL PHASE OF STRUCTURAL DESIGN PROCESS

Although people have been engaged for centuries in engineering design, only in the past few decades has this activity been perceived as a systematic process capable of comprehensive analysis and improvement. The penetration of knowledge-

[1]Asst. Prof., Dept. of Civ. and Envir. Engrg., Univ. of Illinois at Urbana-Champaign, Urbana, IL 62822.

[2]Assoc. Prof., Dept. of Civ. and Envir. Engrg., Massachusetts Inst. of Technol., Cambridge, MA 02139.

Note. Associate Editor: James H. Garrett Jr. Discussion open until November 1, 2000. To extend the closing date one month, a written request must be filed with the ASCE Manager of Journals. The manuscript for this paper was submitted for review and possible publication on February 17, 1999. This paper is part of the ***Journal of Structural Engineering***, Vol. 126, No. 6, June, 2000. ©ASCE, ISSN 0733-9445/00/0006-0733–0742/$8.00 + $.50 per page. Paper No. 20260.

分布式 MULTI-REASONING MECHANISM TO SUPPORT CONCEPTUAL STRUCTURAL DESIGN

作者：卢西奥·索伊贝尔曼[1]与美国土木工程师学会（ASCE）成员费尼奥斯基因·佩纳-莫拉[2]

摘要：结构设计的概念阶段涉及初步材料的选择、建筑总体结构形式的确定、粗略的尺寸布置以及对技术可行性的考量。相关决策需基于诸如建筑高度、使用功能、典型活荷载、风速、地震作用、设计基本周期、设计加速度、最大侧向位移、跨度、层高等信息，以及客户的其他要求。在界定并求解设计问题的过程中，有关任务本身、约束条件、可能的解决方案原则以及类似问题已有解法的更详细信息将极为有益。本文提出了一种名为M-RAM的模型，旨在通过由分布式多推理机制生成的适配型既有设计方案，为高层建筑结构设计的概念阶段提供支持。M-RAM（多推理模型）的目标是以有序且可靠的方式提供这些适配型既有设计方案，从而构建一个辅助系统，以提升设计师的创造力、工程知识与设计经验。为验证所提模型的可行性，开发了一个分布式人工智能系统的原型，并以互联网作为各实施推理机制子系统之间的通信主干。

引言

本文作者认为，设计是一种信息处理活动。在每次信息输出之后，可能都需要对上一步的结果进行改进，并重复各步骤，直至达到所需的技术指标。任何一项设计首先都要求充分理解问题及其已知的相关信息。设计者的知识、能力和经验显然是形成初始设计方案的重要因素；但对于任何一名设计师而言，关于具体任务、约束条件、可行的解决方案原理以及同类问题已有解法的详细信息，都是明确设计需求本质所不可或缺的。

概念设计阶段是设计过程的第一步。在明确问题定义之后，设计师会基于自身的经验与知识、约束条件以及需求，制定一个初步的设计方案。设计过程的效率与成败在很大程度上取决于这一初始方案。由于该设计阶段需要综合运用专业知识、过往经验、经验法则、直觉等多种因素，因此很难采用任何过程式编程语言来实现其计算机化（Maher 1987）。

在概念设计阶段，以往的经验发挥着至关重要的作用；在此过程中，通常通过识别、检索并修改合适的过往设计案例，并对这些案例进行归纳总结来加以辅助，从而隐含地利用相关的过往经验（Manfaat 等，1997）。

本文同时介绍了M-RAM（多推理模型）（Soibelman，1998）以及为评估所提出模型的可行性而开发的一个原型系统。该模型与原型旨在通过向设计人员提供由……生成的、经适配的既有设计解决方案，从而在高层建筑结构设计的初始阶段为工程师提供支持。

[1]Asst. Prof., Dept. of Civ. and Envir. Engrg., Univ. of Illinois at Urbana-Champaign, Urbana, IL 62822.

[2]Assoc. Prof., Dept. of Civ. and Envir. Engrg., Massachusetts Inst. of Technol., Cambridge, MA 02139.

Note. Associate Editor: James H. Garrett Jr. Discussion open until November 1, 2000. To extend the closing date one month, a written request must be filed with the ASCE Manager of Journals. The manuscript for this paper was submitted for review and possible publication on February 17, 1999. This paper is part of the ***Journal of Structural Engineering***, Vol. 126, No. 6, June, 2000. ©ASCE, ISSN 0733-9445/00/0006-0733–0742/$8.00 + $.50 per page. Paper No. 20260.

分布式多推理机制。最终目标是帮助设计人员借鉴以往经验，避免重蹈覆辙。在功能、建筑及美学需求的基础上，M-RAM能够生成若干种结构体系的可行方案，这些方案主要基于重力荷载与侧向荷载（如恒载、活载、风荷载及地震荷载）的几何特征与荷载分布情况。

概念设计是一个复杂的问题，仅靠单一的推理机制难以解决。面对这一挑战，设计师通常综合运用启发式案例、第一性原理以及过往经验，以降低问题的复杂度并提出可行的解决方案。与构建通用功能模块不同，M-RAM模型采用类Agent的方法来开发能力模块，每个模块都专精于某一特定的小型任务。每个模块负责与其特定能力相关的所有表示、计算、推理及执行过程。所提出的架构着重强调感知与行动之间的直接耦合、系统的分布式与去中心化特性、与环境的动态交互，以及应对资源有限性和知识不完备性的内在机制。分布式处理系统在性能、可靠性、模块化和资源共享等方面的传统优势，也为该架构的开发提供了动力（Maes，1991）。应用这一方法，有助于克服传统过程式编程语言在实现高层建筑结构设计概念阶段自动化或辅助设计时所面临的诸多困难。

尽管此前已有多种机器学习算法和推理机制被用于支持概念设计的开发，但用户仍需手动干预这些程序（Maher 和 Fenves，1985；Maher，1987；Fenves 等，1995；Maher 和 de Silva Garza，1996；Manfaat 等，1997）。借助M-RAM，研究重点转向通过计算机网络对现有算法、机制及模型与现有应用软件之间的互联与分布进行考察，从而充分发挥互联网的巨大潜力。

结构设计过程的概念阶段

尽管人类在工程设计领域已从事数个世纪，但直到近几十年来，这一活动才被视作一种能够进行全面分析与改进的系统性过程。知识的渗透……

J. Struct. Eng. 2000.126:733-742.

based tools into the design process is widely perceived as leading to fundamental changes, not only in the accuracy and speed in which a design is achieved, but also in the design process itself.

The engineering design process is primarily a synthesis analysis repetition. The engineer synthesizes (formulates) the problem and then analyzes it. If certain constraints are not met, then the engineer goes back to the synthesis part, changes the formulation, and then analyzes again, and so on, until the specific constraints are met (Pouangare 1986). This synthesis analysis loop is also denoted as the design-analysis cycle.

The conceptual phase of structural design is a typical synthesis problem, and its most important output is the definition of the structural system. The structural system can be defined as an arrangement of structural components and subsystems to properly transmit the forces from the superstructure to the substructure subject to constraints on geometry, flexibility, ductility, durability, and cost. The structural system must have the ability to transmit the forces both globally and locally. In the conceptualization phase, the structural designer is concerned only with the global transmission of forces.

In the early 1970s, computers began to be applied as a helpful tool for structural design, but not until the early 1980s were tools developed to support the synthesis part of the design problem. This led to the situation where the engineer did the synthesis part and the computer did the analysis part of the design (Pouangare 1986).

In the mid-1980s, with the growth of the science of artificial intelligence, researchers initiated attempts to automate the synthesis part of the design problem. Different expert systems were developed. One of the most innovative, called HI-RISE, was developed by Maher and Fenves from Carnegie-Mellon University (Maher and Fenves 1985). Like most expert systems developed at that time, HI-RISE fell short in the learning process. Knowledge acquisition and lack of flexibility were the main problems faced by expert systems such as HI-RISE, and few of these models have been implemented as commercial tools.

In the 1990s, with the development of new artificial intelligence tools, research efforts were initiated to automate the synthesis part of the conceptual phase of structural design. Work done by Fenves, Rivard, Gomez, and Chiou (Fenves et al. 1995) applied case-based reasoning to allow designers to save solutions as a byproduct of the design process. Such cases could then be retrieved in later ''similar'' design situations and adapted to the new situation. Bailey and Smith (1994) used case-based design in the CADRE system as an approach for integrating preliminary architectural and structural design. CADRE retrieves a case in which design integration has already been achieved, and the design process is therefore one of adapting the case to achieve new goals. Maher and de Silva Garza (1996) went one step further, applying genetic algorithms to adapt cases retrieved by the case-based reasoning tool. During this decade, several distributed applications, environments, and testbeds were developed. Lander and Lesser (1992) developed TEAM, a multiple-agent system that supports the integration of heterogeneous and reusable agents; Talukdar, DeSouza, Quadrel, and Ramesh (1992) developed A-TEAM, a multiple-agent organization for cyclic (iterative) processing that has been applied to solve high-rise building design; Killicote and Garrett (1998) developed ASPEN, an agent-based framework for providing computer-aided support for using design standards; and Gasser and Hill (1990) developed MACE, a language, programming environment, and testbed for distributed artificial intelligence systems.

The M-RAM research has built upon the work done by those researchers. The main difference resides in the approach. While the main goal of those researchers was to support agent collaboration, aiding the different participants in the design process, the goal for M-RAM was to study the feasibility of applying agents to support the subprocesses of a divided structural design process. It is recognized that complex problems can be solved by breaking them into smaller problems. Dividing a problem into its smallest unit parts has the objective of imitating the way that the human brain deals with complex problems. According to Minsky (1988), each mind is made of many smaller processes, called agents. Each mental agent by itself can only do some simple thing that needs no thought process at all. Yet when those agents are joined in societies—in certain very special ways—this leads to true intelligence (Minsky 1988). Like the human brain when dealing with complex problems, M-RAM uses different reasoning mechanisms to solve each part of the divided design problem. First, a model was developed to allow a better understanding of the problem requirements. This led to a distributed agent architecture implementation providing a ''plug and play'' approach with easy system maintenance and evolution, as will be demonstrated in the next two sections of this paper.

M-RAM

The M-RAM (Fig. 1) is represented using Unified Modeling Language (UML) (Rational Software Corporation 1997) object-oriented modeling. The model describes the structure of the objects in the system and their identity, relationships to other objects, attributes, and operations. It demonstrates the way in which the multi-reasoning distributed artificial intelligent system selects the characteristics of artifacts such as the structural material, form, system, and parameters for tall building design. The model does so by first choosing the most suitable structural system using a classification agent (C4.5), then supplying different possible structural forms and coefficients from cases provided by the case-based reasoning (CBR) agent, and finally by adapting the retrieved cases by applying a genetic algorithm (GA) agent.

Designer

A designer represents the entity (human, e.g., a structural engineer; or a computer, e.g., a structural engineer assistant) that presents a proposal based on a design intent and/or an artifact characteristic that needs to be satisfied. An example would be a designer who, after receiving information on and specifications from the architect, for the building to be built, defines structural coefficients such as loads (e.g., live, wind, and earthquake), design fundamental period, and maximum lateral deflection based on past experience, local standards, and some structural formulae. All this information is used as input for the next structural design stage: the dimensioning of the structural components.

Proposal

A proposal represents the statement given by the computer designer. A proposal includes two important elements: (1) The recommendation believed to satisfy a design intent and required artifact characteristics; and (2) the justifications for why the recommendation fulfills that design intent and/or those artifact characteristics.

Intent

Design intent refers to what the designer needs to achieve or satisfy (Peña-Mora et al. 1995). Design intent has a ranking that specifies its importance and a satisfaction measure that specifies the extent to which the design intent has been satisfied by a recommendation. Design intent refers to the follow-

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

将基于模型的工具引入设计流程，被普遍认为不仅会从根本上改变设计的精度与效率，还会对设计过程本身产生深远影响。

工程设计过程主要是一个综合与分析的反复迭代过程。工程师首先对问题进行综合（即提出问题的初始方案），然后对其进行分析。若某些约束条件未能满足，则工程师会返回到综合环节，调整问题的表述方式，随后再次进行分析，如此循环往复，直至所有既定约束条件均得到满足（Pouangare，1986）。这一综合—分析的循环也被称作设计—分析循环。

结构设计的概念阶段是一个典型的综合设计问题，其最重要的输出是结构体系的确定。结构体系可定义为在满足几何、刚度、延性、耐久性和成本等约束条件下，将上部结构的荷载合理传递至下部结构的构件及子系统布置方案。该结构体系必须具备全局与局部双重的荷载传递能力。在概念化阶段，结构设计师仅关注荷载的全局传递。

20世纪70年代初，计算机开始被用作结构设计的辅助工具，但直到80年代初才开发出能够支持设计问题中综合阶段的工具。这导致了这样一种局面：工程师负责设计的综合部分，而计算机则承担设计的分析部分（Pouangare，1986）。

20世纪80年代中期，随着人工智能学科的兴起，研究人员开始尝试对设计问题中的综合环节进行自动化。在此期间，多种专家系统相继被开发出来。其中最具创新性的系统之一是HI-RISE，由卡内基梅隆大学的Maher和Fenves研制（Maher & Fenves, 1985）。与当时开发的大多数专家系统一样，HI-RISE在学习能力方面存在不足。知识获取困难以及灵活性欠缺是HI-RISE等专家系统面临的主要问题，因此这些模型中仅有极少数被实际转化为商用工具。

在20世纪90年代，随着新型人工智能工具的发展，研究工作开始致力于实现结构设计概念阶段中方案生成部分的自动化。Fenves、Rivard、Gomez和Chiou等人（Fenves et al., 1995）的研究将基于案例的推理方法应用于设计过程，使设计师能够将解决方案作为设计过程的副产品加以保存。这些案例随后可在后续“相似”的设计情境中被检索，并针对新情境进行适应性调整。Bailey和Smith（1994）在CADRE系统中采用基于案例的设计方法，以实现建筑与结构初步设计的一体化。CADRE系统会检索一个已完成设计集成的案例，因此其设计过程实质上是对该案例进行调整以达成新的设计目标。Maher和de Silva Garza（1996）更进一步，将遗传算法应用于由基于案例的推理工具所检索案例的适应性调整。在同一十年间，还开发了若干分布式应用、环境及实验平台。Lander和Lesser（1992）构建了TEAM系统，这是一种支持异构且可重用智能体集成的多智能体系统；Talukdar、DeSouza、Quadrel和Ramesh（1992）开发了A-TEAM，这是一种用于循环（迭代）处理的多智能体组织架构，已应用于高层建筑设计问题的求解；Killicote和Garrett（1998）研制了ASPEN，这是一个基于智能体的框架，旨在为设计标准的应用提供计算机辅助支持；Gasser和Hill（1990）则开发了MACE，这是一种面向分布式人工智能系统的语言、编程环境及实验平台。

M-RAM研究是在上述研究人员工作的基础上开展的，其主要差异在于研究方法。尽管这些研究人员的主要目标是支持代理间的协作，以协助设计过程中的不同参与者，但M-RAM的研究目标则是探讨将代理技术应用于支持分解后的结构设计过程各子过程的可行性。人们普遍认识到，复杂问题可以通过将其分解为若干较小的问题来求解。将一个问题分解为其最小的单元部件，旨在模拟人脑处理复杂问题的方式。根据明斯基（1988）的观点，每个心智均由众多较小的处理单元构成，这些单元被称为“代理”。单个心智代理本身只能执行一些无需任何思维过程的简单任务；然而，当这些代理以某种特定且极为特殊的方式组合成群体时，便能产生真正的智能（明斯基，1988）。正如人脑在处理复杂问题时所采用的方式一样，M-RAM也利用不同的推理机制来解决被分解的设计问题的各个部分。首先，构建了一个模型，以更好地理解问题的需求。在此基础上，实现了一种分布式代理架构，该架构采用“即插即用”的设计理念，便于系统的维护与演进，这一点将在本文的后两节中予以说明。

M-RAM

M-RAM（图1）采用统一建模语言（UML，Rational Software Corporation，1997）进行面向对象建模。该模型描述了系统中各对象的结构及其标识、与其他对象之间的关系、属性以及操作。它展示了多推理分布式人工智能系统如何为高层建筑设计选择构件的特征，包括结构材料、形式、体系及参数等。具体而言，该模型首先通过分类代理（C4.5）选取最适宜的结构体系，随后由基于案例的推理（CBR）代理提供多种可能的结构形式与相关系数，并最终借助遗传算法（GA）代理对检索到的案例进行调整与优化。

设计者

设计者是指根据设计意图和/或需满足的构件特性提出设计方案的主体（可以是人，如结构工程师；也可以是计算机，如结构工程师辅助系统）。例如，某位设计者在接收到建筑师提供的建筑信息及技术规格后，会依据自身经验、当地规范以及相关结构计算公式，确定诸如荷载（如活荷载、风荷载、地震荷载）、设计基本周期和最大侧向位移等结构参数。所有这些信息将作为下一阶段结构设计——即结构构件尺寸确定——的输入。

提案

提案由计算机设计师提出，包含两个重要要素：（1）被认为能够满足设计意图及所需制品特性的建议；以及（2）该建议为何能够实现该设计意图和/或符合这些制品特性的论证依据。

意图

设计意图是指设计师需要实现或满足的目标（参见 et al., 1995）。设计意图具有一个用于表征其重要性的排序 ˜Pena-Mora，以及一个用于量化某项建议对设计意图满足程度的满意度度量。设计意图指的是以下内容——

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by Tsinghua University on 03/27/20. Copyright ASCE. For personal use only; all rights reserved.

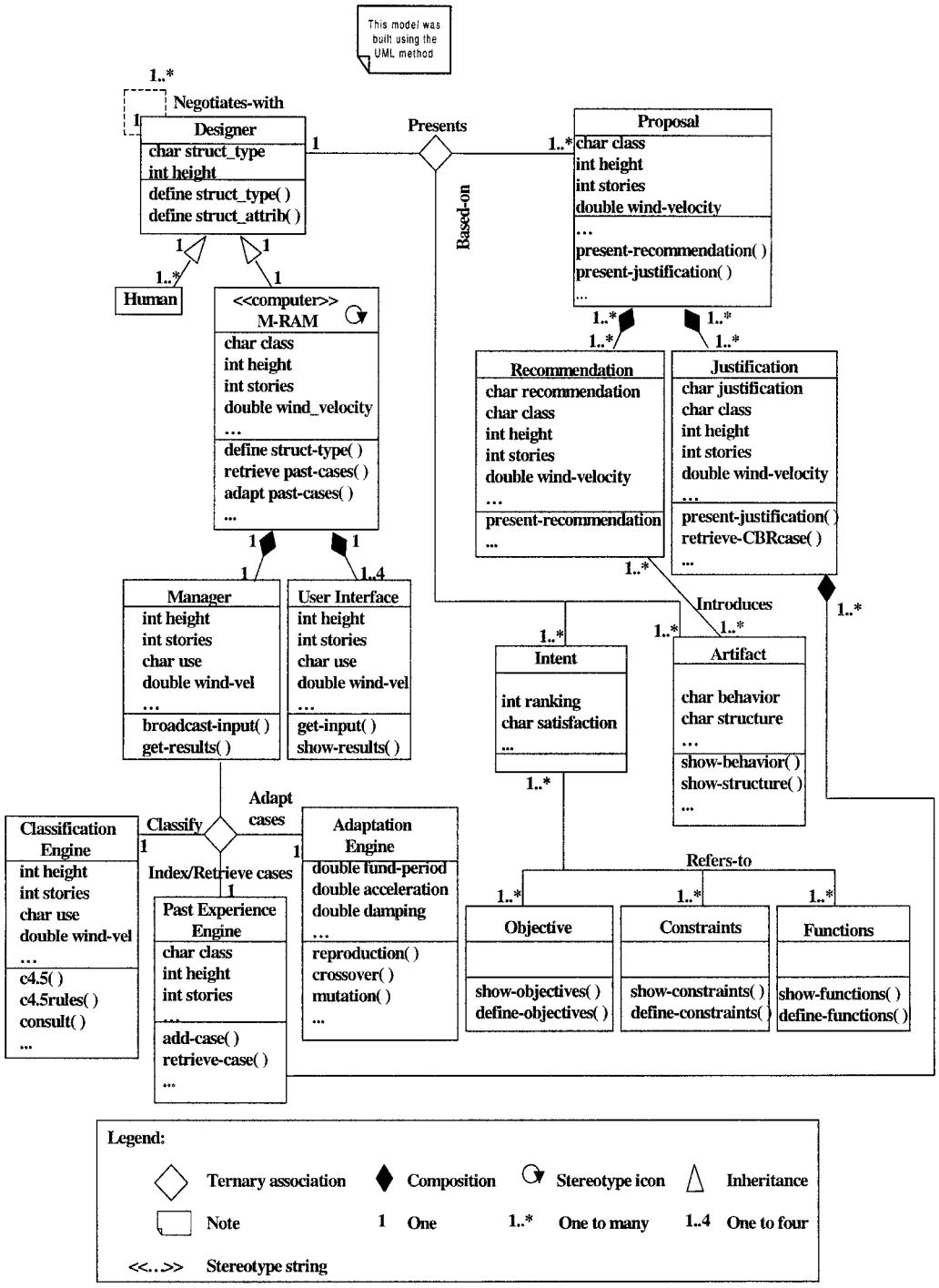


FIG. 1. M-RAM Object Model. Graphical Notation of UML Model (Rational Software Corp. 1997)

ing attributes of a design: (1) objectives; (2) constraints; and (3) functions.

- ''Objective'' is a characteristic to be optimized by the artifact. It presents a measure against which the design is checked (e.g., minimize the construction cost). Designers tend to use this class of intents as an evaluation, which requires comparison among several competing designs.
- ''Constraints'' is a confinement of an artifact (e.g., lack of skilled labor in certain regions, making it very difficult to build concrete structures). In this class of intents, designers only need to test if the criterion is met by the design recommendation without the need to compare it to other design alternatives.
- ''Function'' is an activity performed by an artifact (e.g., to safely withstand the earthquake loads during the building lifetime). This class establishes the performance criteria that later translates to the behavior of the system and specific constraints.

Recommendation

Recommendation refers to the entity that satisfies the design intents and/or required artifact characteristics. A recommendation is a proposed structural system presented by the classification agent, the cases presented by the CBR agent, and the adaptation of the retrieved cases presented by the GA agent. Several feasible structural systems could be proposed for the same tall building.

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

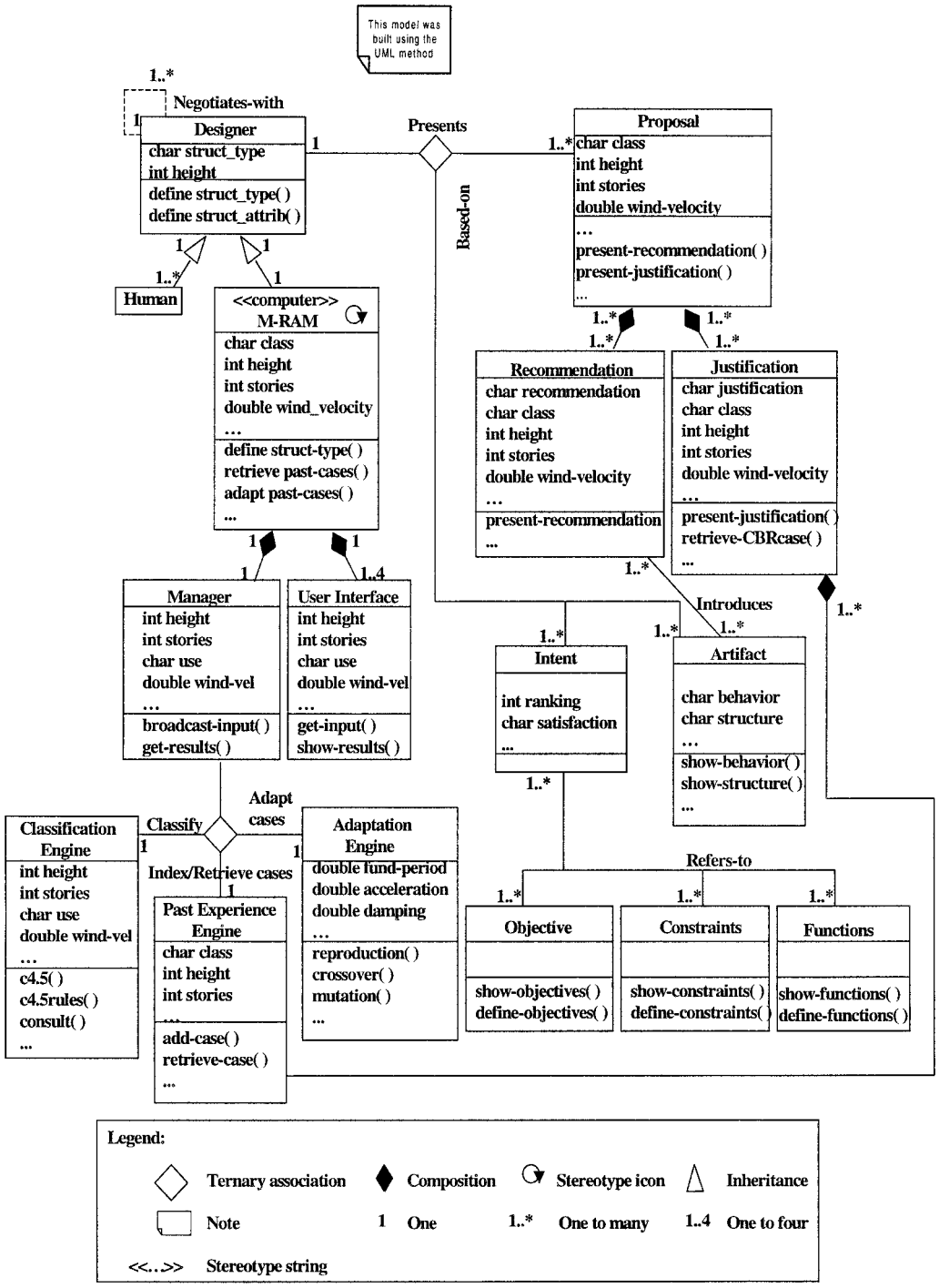


图1. M-RAM对象模型。UML模型的图形表示法（Rational Software Corp.，1997年）

设计的三个基本属性：（1）目标；（2）约束；以及（3）功能。

- “目标”是待优化的制品所具有的特性，它提供了一种用于检验设计方案的度量标准（例如，使建造成本最小化）。设计人员通常将此类意图用作评价依据，这需要在多个竞争性设计方案之间进行比较。
- “约束”是指对某一构件的限制条件（例如，某些地区缺乏熟练劳动力，导致建造混凝土结构极为困难）。在这一类意图中，设计人员只需检验设计方案是否满足该约束条件，而无需将其与其他设计方案进行比较。
- “功能”是指由某一构件所执行的活动（例如，在建筑物的使用寿命期内安全地承受地震荷载）。该类别确立了性能标准，这些标准随后转化为系统的性能表现及具体的约束条件。

建议

推荐是指能够满足设计意图和/或所需构件特性的实体。推荐包括分类代理所提出的结构体系、案例推理代理所呈现的案例，以及遗传算法代理对检索到的案例所进行的适应性调整。对于同一座高层建筑，可能会提出多个可行的结构体系方案。

J. Struct. Eng. 2000.126:733-742.

Justification

Justification refers to the reason that partially explains why a recommendation will satisfy a design intent. According to Peña-Mora (1994), a justification could be a rule (e.g., if the number of floors <40, then a braced-frame structural system is feasible); a case (e.g., this building is similar to the Sears Tower in Chicago); a catalogue (e.g., this structural system solution was taken from an entry in the *Structural Systems for Tall Buildings* published by the Council on Tall Buildings and Urban Habitat (Kowalczyk et al. 1995); a principle (e.g., Hooke's law); an authority (e.g., this structural system has been suggested by someone who is an authority in the field); a trade-off (e.g., this is the best possible design based on the trade-off between minimizing the cost and minimizing the deflection); a prototype (e.g., the model of the proposed structure produced these measurements in the wind tunnel); a constraint network (e.g., this condition satisfies all the constraints on the system); or a Pareto optimal surface (e.g., this design falls on the surface of the best possible design when optimizing cost, schedule, and ecological impact). A justification may support other justifications by providing supporting evidence or assumptions. Cases of past experience provided by the CBR engine are used as justification in the M-RAM implementation.

Computer—M-RAM

The M-RAM computer implementation is a distributed artificial intelligence computer system developed as a decision support tool for the preliminary design of tall building structures. Decisions are made on the basis of such information as height of the building, building use, region, typical live load, wind velocity, earthquake loading, design fundamental period, design acceleration, maximum lateral deflection, spans, story height, and other client requirements. It runs on the Internet using a World Wide Web (WWW) interface.

Manager and User Interface

The Internet is being used as a communication backbone for the different systems that implement the reasoning mechanisms being employed. Each reasoning mechanism considered as an autonomous module can be installed in different servers. Each distributed server acts as an intelligent agent. A WWW interface receives the expected data as input and launches a manager program, which broadcasts the problem to all existing modules. Each module solves a part of the problem and sends back its partial solution to the manager program. This programs sends the solution or possible solutions back to the WWW interface, which displays it to the user. In the current implementation, M-RAM was developed with three modules: the classification module implemented by the C4.5 agent; the past cases/experience module implemented by the CBR agent; and the adaptation module implemented by the GA agent.

Classification Engine

In the M-RAM implementation, a C4.5 agent (Quinlan 1993) implemented the classification engine. In C4.5, recorded classifications are examined and a model is constructed inductively by generalization from specific examples. This machine-learning algorithm can be defined as a set of computer programs that construct classification models by discovering and analyzing patterns found in given records. The algorithm that has fundamental importance in C4.5 is the one that generates the initial decision tree from a set of training cases. Since the cases do not all belong to the same class, a divide-and-conquer algorithm attempts to split them into subsets using statistical evaluation tests and an information gain ratio criterion to find the best possible tree structure (Quinlan 1993).

The C4.5 agent can be divided into a set of training cases, a decision tree, and a set of decision rules generated by simplification of the decision tree. The decision tree can be subdivided into leaves, which indicate classes, and decision nodes, which specify some test to be carried out on a single attribute value (Fig. 2).

For the practicing structural engineer, the cataloguing of structural systems for tall buildings has historically recognized the primary importance of the system to resist lateral loads (Kowalczyk et al. 1995). At one end of the spectrum are the moment-resisting frames, which are efficient for buildings in the range of 20 to 30 stories; at the other end is the generation of a tubular system with high cantilever efficiency that is good for 70 to 80 stories. Any particular form is economical for a limited range of building heights.

Recognizing this historical knowledge, M-RAM applies C4.5 to create a decision tree and a set of rules with the objective of clustering the database and consequently reducing the search space. The database, obtained from the monographs edited by the Council on Tall Buildings and Urban Habitat (Kowalczyk et al. 1995), was classified into five classes according to the system adopted to resist lateral loads: braced frame and moment-resisting frame, shear wall, core and outrigger, tubular, and hybrid. This class division was defined by the available cases. The M-RAM classification agent is flexible enough to allow the creation of any other category, according to future available cases.

In the M-RAM C4.5 implementation, each case concerns one design. Nineteen properties have been used to describe each case: (1) height from street to roof; (2) number of stories; (3) number of levels below ground; (4) building use; (5) frame material; (6) typical floor life load; (7) basic wind velocity; (8) maximum lateral deflection; (9) design fundamental period—transverse; (10) design fundamental period—longitudinal; (11) design acceleration; (12) design damping; (13) earthquake loading; (14) typical floor story height; (15) typical floor beam span; (16) typical floor beam depth; (17) typical floor beam spacing; (18) column size at ground floor; and (19) column spacing. The outcome of each case is given as braced frame and moment-resisting frame, shear wall, core and outrigger, tubular, or hybrid, depending on which kind of structural system was applied to resist lateral loads.

From the available cases, C4.5, acting as a learning system, constructed a classification model that relates the type of structural system to resist lateral loads to the values of the recorded properties. There are 46 cases of actual built tall buildings in the dataset. In the implementation, the same database is used to build the decision tree model and to create the past experience case database. We recognize the fact that these data are not sufficient to induce a good generalization. A robust pattern cannot be distinguished from chance coincidences, but the objective in this stage of the research was to define the viability of the proposed system architecture. The next step in this research is to augment the dataset; consulting engineers and computer simulation can be used to accomplish this.

Past Experience Engine

A case-based reasoning agent implemented the past experience engine. Case-based reasoning is the process of ''remember and compare'' (Kolodner 1993). A reasoner remembers previous situations similar to the current one and uses them to help solve a new problem. The knowledge of a case-based reasoner is constantly changing as new experience gives rise to new cases that are stored for future use. A case-based reasoner learns from experience to exploit prior successes and avoid repeating past failures (Leake 1996).

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

正当性

理由是指能够部分解释某项建议为何能满足设计意图的原因。根据1994年的文献，理由可以是规则（例如：若楼层总数˜Pena-Mora <40，则采用支撑框架结构体系是可行的）；案例（例如：该建筑与芝加哥的西尔斯大厦相似）；目录（例如：该结构体系方案取自由高层建筑与城市栖息地委员会发布的*Structural Systems for Tall Buildings*中的某条记录（Kowalczyk等，1995））；原理（例如：胡克定律）；权威依据（例如：该结构体系系由本领域权威人士所推荐）；权衡结果（例如：在成本最小化与变形最小化之间的权衡下，这是最优的设计方案）；原型（例如：所提结构模型在风洞试验中得出这些测量结果）；约束网络（例如：该条件满足系统的所有约束）；或帕累托最优面（例如：在同时优化成本、进度和生态影响时，该设计方案位于最优设计曲面上）。一个理由可以通过提供支持性证据或假设来支撑其他理由。在M-RAM的实现中，基于案例推理引擎所提供的过往经验案例被用作理由。

计算机—磁阻随机存取存储器

M-RAM计算机实现是一种分布式人工智能计算机系统，被开发用作高层建筑结构初步设计的决策支持工具。决策依据的信息包括建筑高度、建筑用途、所在地区、典型活荷载、风速、地震作用、设计基本周期、设计加速度、最大侧向位移、跨度、层高等参数，以及客户的其他要求。该系统通过万维网（WWW）界面在互联网上运行。

管理界面与用户界面

互联网被用作承载不同系统通信的骨干网络，这些系统实现了所采用的推理机制。每个被视为自主模块的推理机制均可部署于不同的服务器上，每台分布式服务器均作为智能代理运行。一个基于万维网的界面接收预期输入数据，并启动一个管理程序，该程序将问题广播至所有已存在的模块。各模块分别求解问题的一部分，并将其局部解反馈给管理程序；管理程序再将最终解或可能的解返回至该万维网界面，由其呈现给用户。在当前实现中，M-RAM系统由三个模块构成：由C4.5代理实现的分类模块、由CBR代理实现的历史案例/经验模块，以及由GA代理实现的适应模块。

分类引擎

在M-RAM的实现中，一个C4.5智能体（Quinlan，1993）负责执行分类引擎的功能。在C4.5算法中，通过对已记录的分类结果进行分析，并基于具体实例进行归纳概括，从而构建模型。该机器学习算法可被定义为一组计算机程序，其通过发现并分析给定数据记录中的模式来构建分类模型。在C4.5中具有核心意义的算法是从一组训练样本中生成初始决策树的过程。由于这些样本并不全部属于同一类别，因此采用分而治之的算法试图借助统计方法将它们划分为若干子集。采用评估测试和信息增益比准则来确定最优的树结构（Quinlan，1993）。

C4.5代理可分为一组训练样本、一棵决策树以及通过简化该决策树而生成的一组决策规则。决策树又可进一步划分为表示类别的叶节点和用于对单个属性值进行某种测试的决策节点（图2）。

对于从事结构工程实践的工程师而言，高层建筑结构体系的分类历来强调该体系在抵抗侧向荷载方面的首要作用（Kowalczyk 等，1995）。在这一谱系的一端是抗弯框架体系，其适用于20至30层左右的建筑；而在另一端则是具有高悬臂效率的筒体体系，适用于70至80层左右的建筑。任何一种具体形式仅在有限的建筑高度范围内具有经济性。

鉴于这一历史知识，M-RAM采用C4.5算法构建决策树并生成规则集，旨在对数据库进行聚类，从而缩小搜索空间。该数据库来源于由高层建筑与城市栖息地委员会编辑的专著（Kowalczyk等，1995），并根据其抵抗侧向荷载的体系被划分为五类：支撑框架与抗弯框架、剪力墙、核心筒与外伸桁架、筒体结构以及混合结构。这一分类体系是依据现有案例确定的；M-RAM分类代理具有足够的灵活性，可根据未来新增的案例随时扩展为其他类别。

在M-RAM C4.5的实现中，每个案例对应一种设计方案。每个案例采用十九个属性进行描述：（1）从街道到屋面的高度；（2）楼层数；（3）地下层数；（4）建筑用途；（5）框架材料；（6）典型楼层活荷载；（7）基本风速；（8）最大侧向位移；（9）横向设计基本周期；（10）纵向设计基本周期；（11）设计加速度；（12）设计阻尼比；（13）地震作用；（14）典型层高；（15）典型楼层层间梁跨度；（16）典型楼层层间梁截面高度；（17）典型楼层层间梁间距；（18）首层柱截面尺寸；以及（19）柱间距。每个案例的输出结果为支撑框架、抗弯框架、剪力墙—核心筒与外伸桁架体系、筒体结构或混合结构，具体取决于所采用的用于抵抗侧向荷载的结构体系类型。

根据现有案例，C4.5作为一种学习系统，构建了一个分类模型，该模型将抗侧力结构体系的类型与所记录的各项性能指标相关联。数据集中包含46栋实际建造的高层建筑案例。在具体实施过程中，同一数据库既用于构建决策树模型，也用于建立历史经验案例库。我们认识到，这些数据尚不足以实现良好的泛化能力；难以将稳健的模式与偶然性区分开来，但本研究阶段的目标在于验证所提出的系统架构的可行性。下一步研究计划是扩充数据集，可通过咨询工程师和计算机仿真来实现这一目标。

过往经验引擎

基于案例的推理代理实现了过往经验引擎。基于案例的推理是一种“记忆与比较”的过程（Kolodner，1993）。推理机能够回忆起与当前情境相似的先前情境，并利用这些情境来辅助解决新问题。随着新经验的积累，基于案例的推理机的知识库会不断更新，新的案例被存储以供未来使用。基于案例的推理机通过经验学习，以充分利用以往的成功并避免重蹈覆辙（Leake，1996）。

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by Tsinghua University on 03/27/14. Copyright ASCE. For personal use only; all rights reserved.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

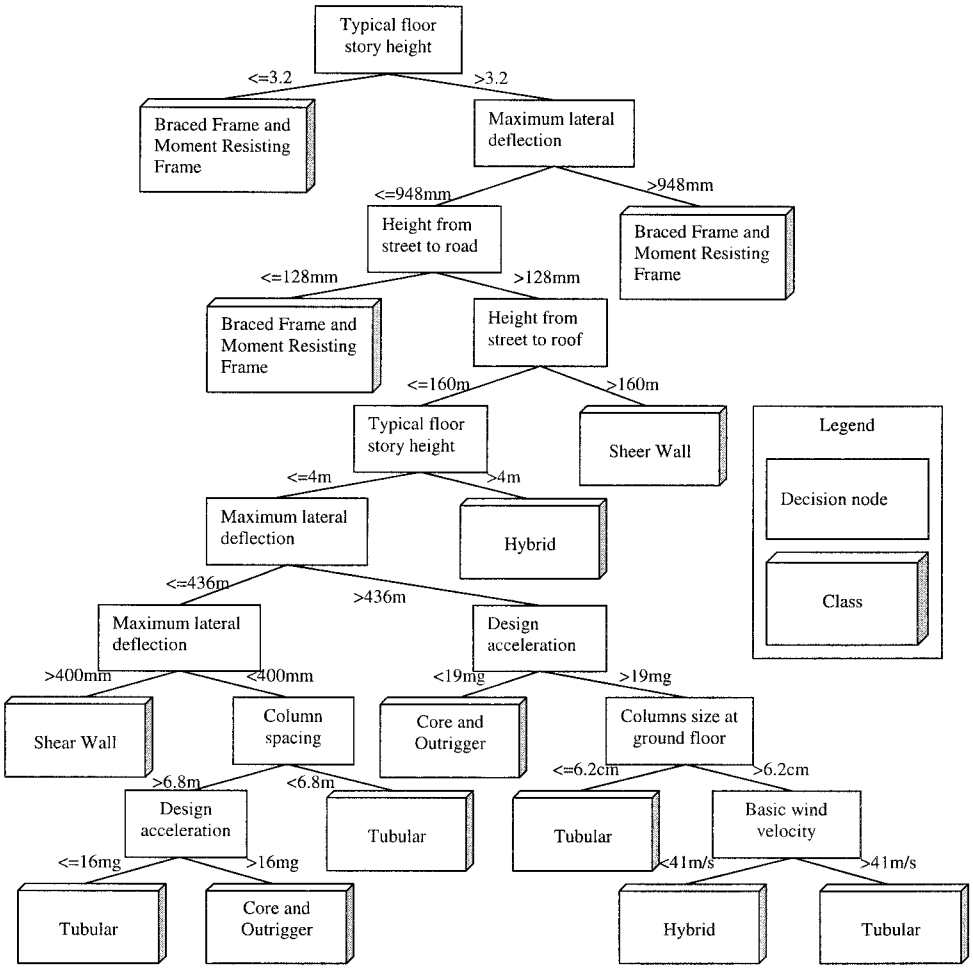


FIG. 2. M-RAM C4.5 Tree

The CBR agent can be divided into three parts: a case library, an indexing engine, and a retrieving engine. The case library contains the case problem description (e.g., the state of the world at the moment the case was happening and, if appropriate, what problem needed solving at the time), the case solution description (e.g., the stated or derived solution to the problem specified in the problem description, or the reaction to the situation), and the case outcome (e.g., the resulting state of the world after the solution was carried out). The indexing engine can be subdivided into two parts: the case-labeling engine and the library organization engine. The case-labeling engine ensures that cases can be retrieved at appropriate times. In general, labels designate under what circumstances the case might have a lesson to teach and therefore when it is likely to be useful. The library organization engine organizes the cases so that the search through the case library can be done efficiently and accurately. Finally, the retrieving engine can be subdivided into the searching engine (e.g., finding potentially matching cases) and a matching engine (e.g., each case is judged for its potential usefulness).

In the M-RAM implementation, the manager, after receiving the classification from the C4.5 agent, requests the CBR agent to retrieve the most useful case. The CBR agent returns to the manager the home page of the retrieved case. From this page, the designer can follow the links to complete information about the retrieved case.

Caspian (''Introductory'' 1996b), a case-based reasoning system written in C and developed at the University of Wales —Aberystwith, was chosen to be implemented as the M-RAM CBR agent. Caspian creates a case base from a case file written in the language CASL (''Creating'' 1996a). The contents of a case base are described in a file known as a case file. The program Caspian uses this case file to create a case base in the computer memory, which can then be accessed and modified to retrieve the most suitable design solution to the problem being solved using case-based reasoning techniques. When new cases are added to the case base, they are appended to the end of this case file.

Caspian requires the cases to be indexed according to one or more attributes. The program searches the case base for the subset of cases that matches all the index constraints exactly. In the M-RAM implementation, the cases were indexed according to their structural system type. The program then searches the case base for all cases that match the class previously defined by the M-RAM classification agent.

After the index search has been completely successfully, Caspian can be run in two different modes: with or without case matcher. If the case matcher is invoked, the program scans the subset of cases to find the one with the highest weight value. The weight is calculated using the same 19 properties that were used to describe each case in the classification agent. If the case matcher is not invoked, the program retrieves the whole subset of cases that matches the indexed constraints. In the M-RAM implementation, Caspian is run in both modes. First, the program invokes the case matcher to retrieve the case that is the best fit to the design problem being analyzed; then the program is run again without the case matcher, with the objective of retrieving all cases that match the class defined by the M-RAM's classification agent. All retrieved cases are then used as the initial generation in the M-RAM adaptation agent.

J. Struct. Eng. 2000.126:733-742.

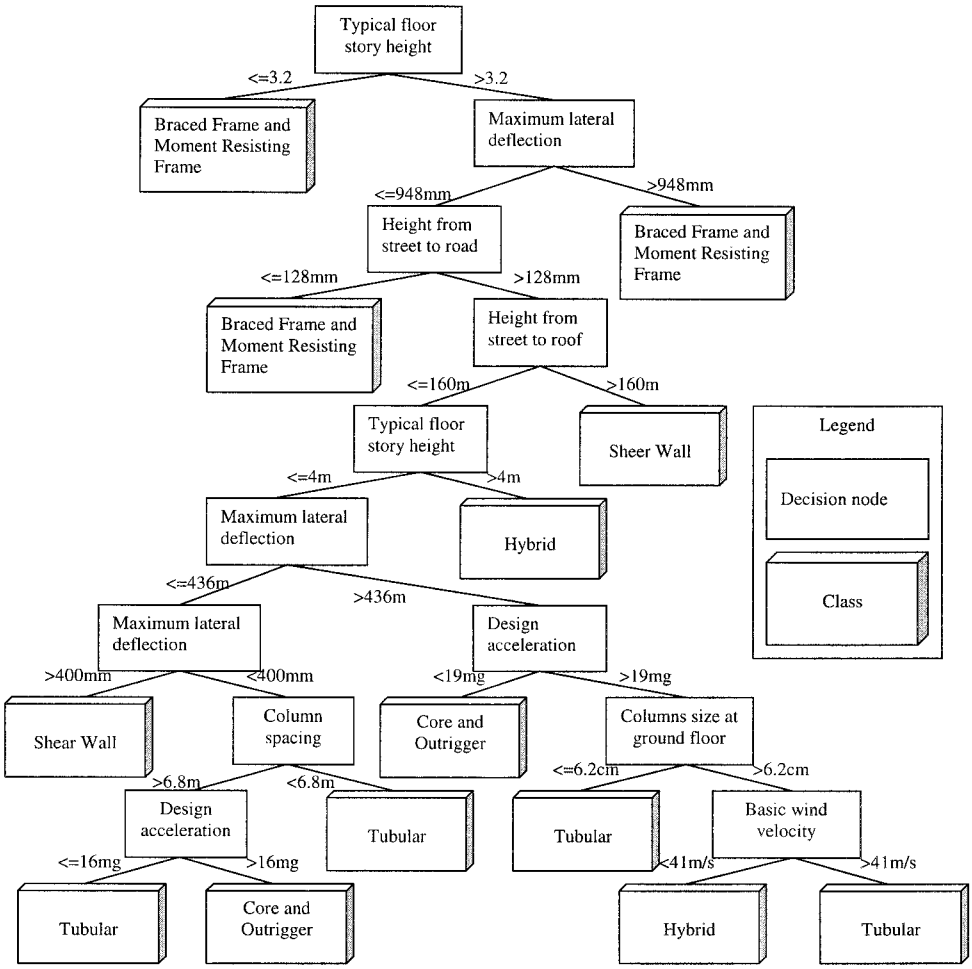


图2. M-RAM C4.5决策树

CBR 代理可分为三个部分：案例库、索引引擎和检索引擎。案例库中存储着案例的问题描述（例如，案例发生时的世界状态，以及在适当情况下当时需要解决的具体问题）、案例的解决方案描述（例如，针对问题描述中所指明问题的既定或推导出的解决方案，或对该情境的应对措施），以及案例的结果（例如，在实施解决方案后世界状态的变化情况）。索引引擎又可细分为两个子模块：案例标注引擎和库组织引擎。案例标注引擎负责确保在适当的时间能够检索到相应的案例；通常，标签用于标识该案例在何种情境下可能具有借鉴意义，从而确定其适用时机。库组织引擎则对案例进行有序组织，以实现对案例库的高效且准确的检索。最后，检索引擎也可进一步划分为搜索引擎（例如，查找潜在匹配的案例）和匹配引擎（例如，对每个案例的潜在有用性进行评估）。

在M-RAM的实现中，管理器在接收到C4.5智能体的分类结果后，会请求CBR智能体检索最相关的案例。CBR智能体将所检索案例的概览页面返回给管理器。设计人员可通过该页面中的链接获取关于该案例的完整信息。

Caspian（“简介”1996b）是一款基于案例的推理系统，采用C语言编写，并由威尔士大学阿伯里斯特威斯分校开发。该系统被选定作为M-RAM CBR代理的实现方案。Caspian通过解析以CASL语言编写的案例文件来构建案例库。（“创建”1996a）。其中的内容……

案例库以一种称为“案例文件”的文件形式进行描述。Caspian程序利用该案例文件在计算机内存中构建案例库，随后可通过访问和修改该库，借助基于案例的推理技术检索出最适合当前待解决问题的设计方案。当有新案例被添加至案例库时，这些案例将被追加到该案例文件的末尾。

Caspian 要求按照一个或多个属性对案例进行索引。该程序会在案例库中搜索与所有索引约束完全匹配的案例子集。在 M-RAM 的实现中，案例是根据其结构体系类型进行索引的。随后，程序会在案例库中搜索所有与 M-RAM 分类代理先前定义的类别相匹配的案例。

在索引检索完全成功之后，Caspian 可以以两种不同的模式运行：启用或禁用案例匹配器。若启用案例匹配器，程序将对案例子集进行扫描，以找出权重值最高的案例。该权重值的计算采用与分类代理中用于描述每个案例的同一组19个属性。若未启用案例匹配器，程序则会检索满足已索引约束条件的整个案例子集。在M-RAM的实现中，Caspian 会同时以这两种模式运行：首先调用案例匹配器，以检索与当前所分析设计问题最为匹配的案例；随后再次运行程序，但不启用案例匹配器，旨在检索所有符合M-RAM分类代理所定义类别要求的案例。所有检索到的案例随后将作为M-RAM适应代理中的初始种群。

J. Struct. Eng. 2000.126:733-742.

Gene	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Value	211.9	47	2	5	1	5	30	1000	4.52	4.73	52	1.5	0.06	3.58	17.5	850	6.4	4500	6.4

Where:

0 Height from the street to the roof
1 Number of stories
2 Number of levels below ground
3 Building use: 1=office; 2=hotel; 3=residential; 4=garage; 5=mixed
4 Frame material: 1=steel; 2=concrete; 3=mixed
5 Typical floor life load
6 Basic wind velocity
7 Maximum lateral deflection
8 Fundamental-period transverse
9 Fundamental-period longitudinal
10 Design acceleration
11 Design damping
12 Earthquake loading
13 Typical floor story height
14 Typical floor beam span
15 Typical floor beam depth
16 Typical floor beam spacing
17 Columns size at ground floor
18 Columns spacing

FIG. 3. Building Genome Representation

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

Adaptation Engine

Genetic algorithms (GA) provide a robust yet efficient search methodology, explicitly modeled upon the biological ''survival of the fittest'' reproductive model. In the broadest sense, a GA creates a set of solutions that reproduce based on their fitness in a given environment. The process follows the following pattern: (1) An initial population of random solutions is created; (2) each member of the population is assigned a fitness value based on its evaluation against the current problem; (3) solutions with a higher fitness value are most likely to parent new solutions during reproduction; and (4) the new solution set replaces the old, a generation is complete, and the process continues from step 2. That sequence implements, in a most simplistic way, the concept of the survival of the fittest. The outcome of a genetic algorithm is based on probabilities, just as biological success is grounded in chance. The standard model for a GA solution is a bit string called a chromosome, after its biological counterpart. During reproduction, the chromosomes of parent solutions combine and undergo mutation in creating the next generation.

The GA agent can be subdivided into a phenotype (e.g., a design solution) and a genotype (e.g., a way of representing or encoding the information that is used to produce the phenotype) (Koza 1996). The principle of the survival of the fittest determines whether a genotype survives to reproduce. The first step in preparing to solve a GA problem is the identification of a suitable representation scheme. After the phenotype is converted into a genotype, the GA solves the problem with the help of three basic operations: reproduction, crossover, and mutation.

In the M-RAM implementation, the partially matching retrieved design cases provided by the CBR agent are used as the initial population (i.e., generation 0) for the GA. The retrieved design cases that are represented using attribute-value formalism describe a specific design solution and correspond to the building's genotype. Here it is important to consider that, unlike the traditional implementation of GA, where the initial (e.g., seed) population has a few extraordinary individuals within a population of mediocre colleagues, the M-RAM deals with the above-average initial population provided by the CBR agent. In this GA implementation, what is needed is just a fine tuning. This implies that the M-RAM adaptation engine relies heavily on reproduction and crossover, having a very low mutation rate.

GAlib, a C++ genetic algorithm library developed at MIT, was used to implement M-RAM's GA agent (Wall 1996). The library includes tools for using genetic algorithms to do optimization in any C++ program using any representation and any genetic operators. The GAlib library is centered around two main classes: a genome and a genetic algorithm. Each genome represents a single solution to the problem. The genetic algorithm object defines how the evolution should take place. The genetic algorithm uses an objective function to determine how ''fit'' each genome is for survival. It uses genome operators (e.g., built into the genome) and selection/replacement strategies (e.g., built into the genetic algorithm) to generate new individuals. The genetic algorithm object determines which individuals should survive, which should reproduce, and which should die. It also records statistics and decides how long the evolution should continue.

The objective of the M-RAM's GA agent is to combine bits and pieces from several past experiences retrieved from the CBR agent in order to solve a new problem. The attribute-value representation of cases retrieved by M-RAM's CBR agent is reinterpreted and used in the context of the genetic algorithm by considering the attributes used in the description of a case to be equivalent to genes. Thus, the collection of attributes used to describe a specific building corresponds to the building's genome. The values that are associated with each of the 19 attributes in the description of a case make up the case description and can be defined as the case phenotype (Fig. 3).

The type of genetic algorithm applied by M-RAM's GA agent is the steady-state algorithm implemented by GAlib's GASteadyStateGA class. This genetic algorithm uses overlapping populations with a user-specified amount of overlap. The algorithm creates a population of individuals by cloning the population passed by M-RAM's CBR agent. In each generation the algorithm creates a temporary population of individuals, adds these to the previous population, then removes the worst individuals in order to return the population to its original size. Unlike most genetic algorithm implementations that create a random initial population, this approach allowed the cloning of the cases provided by the CBR agent as an initial population. The amount of overlap could be specified to allow the required fine tuning by maintaining most desired characteristics of the above average cases provided by the CBR agent.

The objective function is an adaptation of the one presented by Maher and de Silva Garza (1996). Two types of considerations are used to measure a solution's fitness in M-RAM's GA agent: the problem requirements, and the general structural design constraints (represented by attribute correlation). Both types of fitness are represented as constraints, and the objective is measured by the number of constraints not violated. The solution is feasible if none of the constraints are violated; therefore, the goal of the GA agent is to maximize constraint nonviolations.

Two examples of constraint are

1. Problem requirement constraint: If 300 m is the required height of the building being designed, the height is then considered to be one of the design constraints. To evaluate a genome proposed by the genetic algorithm, the objective function adds 10 points to the objective value of all genomes that have height in the interval (285–315 m), 5 points to genomes that have height in the intervals (240–285 m) and (315–360 m), and nothing to all other genomes.
2. General structural design constraint: To retrieve feasible

J. Struct. Eng. 2000.126:733-742.

Gene	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Value	211.9	47	2	5	1	5	30	1000	4.52	4.73	52	1.5	0.06	3.58	17.5	850	6.4	4500	6.4

Where:

0 Height from the street to the roof
1 Number of stories
2 Number of levels below ground
3 Building use: 1=office; 2=hotel; 3=residential; 4=garage; 5=mixed
4 Frame material: 1=steel; 2=concrete; 3=mixed
5 Typical floor life load
6 Basic wind velocity
7 Maximum lateral deflection
8 Fundamental-period transverse
9 Fundamental-period longitudinal
10 Design acceleration
11 Design damping
12 Earthquake loading
13 Typical floor story height
14 Typical floor beam span
15 Typical floor beam depth
16 Typical floor beam spacing
17 Columns size at ground floor
18 Columns spacing

图3. 基因组表示的构建

适配引擎

遗传算法（GA）提供了一种稳健且高效的搜索方法，其设计明确仿照了生物界的“适者生存”繁殖机制。从广义上讲，遗传算法通过在给定环境中依据适应度对解进行选择与繁殖，从而生成一组候选解。该过程遵循如下模式：（1）首先生成一个由随机解构成的初始种群；（2）根据各解对当前问题的评估结果，为其分配相应的适应度值；（3）适应度较高的解在繁殖过程中更有可能作为父代产生新的子代解；（4）用新生成的解集替换旧的解集，完成一代迭代，并从步骤2重新开始。这一序列以最简化的形式实现了“适者生存”的核心理念。遗传算法的最终结果同样基于概率，正如生物界的繁衍成功也建立在偶然性之上。遗传算法的标准解表示形式是一种称为“染色体”的二进制字符串，其名称源自生物学中的同名概念。在繁殖过程中，父代解的染色体会发生重组，并在形成下一代时经历突变。

遗传算法个体可进一步划分为表型（例如，一种设计解决方案）和基因型（例如，用于生成该表型的信息的表示或编码方式）（Koza，1996）。适者生存原则决定了某一基因型能否存活并进行繁殖。在准备求解遗传算法问题时，首要步骤是确定合适的表示方案。将表型转换为基因型之后，遗传算法借助三种基本操作——复制、交叉和变异——来求解问题。

在M-RAM的实现中，由CBR智能体检索出的部分匹配设计实例被用作遗传算法的初始种群（即第0代）。这些采用属性-值形式表示的设计实例描述了具体的设计方案，并对应于建筑的基因型。值得注意的是，与传统的遗传算法实现不同——在后者中，初始种群（如种子种群）往往包含少数表现优异的个体，而大多数个体则处于中等水平——M-RAM所处理的是由CBR智能体提供的高于平均水平的初始种群。在这种遗传算法的实现中，所需仅是对种群进行微调。这意味着M-RAM的适应引擎主要依赖于繁殖和交叉操作，而突变率则非常低。

GAlib 是由麻省理工学院开发的 C++ 遗传算法库，用于实现 M-RAM 的遗传算法智能体（Wall，1996）。该库提供了一系列工具，支持在任何 C++ 程序中采用任意编码方式和任意遗传算子来开展优化求解。GAlib 库的核心由两个主要类构成：基因组和遗传算法。其中，每个基因组代表问题的一个候选解；而遗传算法对象则定义了进化过程的具体实施方式。遗传算法通过目标函数来评估各基因组的“适应度”，从而决定其是否能够存活，并借助基因组自身的算子（例如内嵌于基因组中的算子）以及选择/替换机制来推进进化过程。

采用多种策略（例如内置于遗传算法中）来生成新的个体。遗传算法对象负责确定哪些个体应当存活、哪些个体应当繁殖，以及哪些个体应当被淘汰；同时，它还会记录相关统计信息，并决定进化过程应持续多长时间。

M-RAM的遗传算法智能体的目标是，将从案例推理智能体中检索到的若干过往经验片段加以组合，以求解决新问题。M-RAM的案例推理智能体所检索案例的属性-值表示被重新诠释，并在遗传算法的框架下加以运用；其中，用于描述案例的各属性被视为等同于基因。因此，用于表征特定建筑的一组属性即构成该建筑的基因组。与案例描述中19个属性分别对应的取值共同构成了案例的描述，并可定义为案例的表型（图3）。

M-RAM 的 GA 代理所采用的遗传算法类型是基于 GAlib 库中 GASteadyStateGA 类实现的稳态遗传算法。该算法采用重叠种群，并允许用户指定重叠程度。算法通过克隆由 M-RAM 的 CBR 代理传递的种群来生成初始个体集。在每一代中，算法首先构建一个临时个体集，将其并入上一代种群，随后移除适应度最差的个体，以使种群规模恢复至初始大小。与大多数通过随机方式生成初始种群的遗传算法实现不同，这一方法允许将 CBR 代理提供的案例作为初始种群进行克隆。通过指定适当的重叠程度，可以在保留 CBR 代理所提供的高于平均水平案例的大部分优良特征的同时，实现所需的精细调整。

目标函数是对Maher和de Silva Garza（1996）所提出的函数的改进。在M-RAM的遗传算法代理中，采用两类考量来度量解的适应度：问题需求以及一般性的结构设计约束（由属性相关性表征）。这两类适应度均以约束的形式表达，而目标则通过未被违反的约束数量来衡量。若所有约束均未被违反，则该解为可行解；因此，遗传算法代理的目标是使未被违反的约束数量最大化。

约束的两个示例是

1. 问题需求约束：若所设计建筑的 required 高度为300米，则该高度即被视为设计约束之一。为评估由遗传算法生成的某一基因组，目标函数对所有高度位于区间（285–315米）内的基因组，在其目标值上增加10分；对高度位于区间（240–285米）和（315–360米）内的基因组，各增加5分；而对于其他所有基因组，则不作任何调整。 2. 一般结构设计约束：为了获得可行解

J. Struct. Eng. 2000.126:733-742.

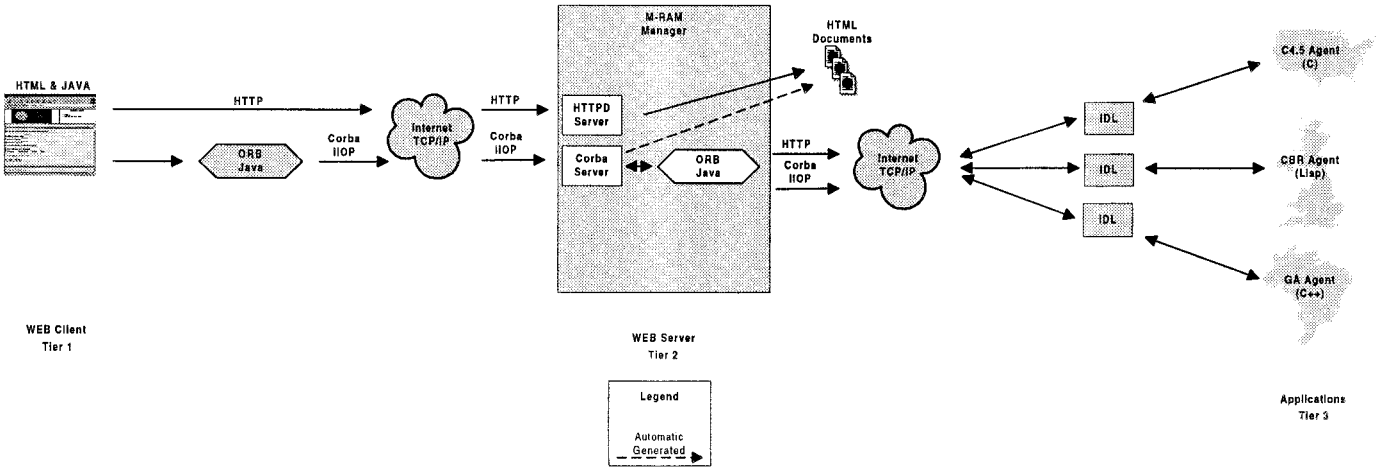


FIG. 4. M-RAM Architecture

design solutions, the objective function gives points to solutions that keep the proportionality among number of floors, story height, and building height. The objective function gives 10 points to all genomes that respect the following equation:

$$0.95 * \text{Story Height} < (\text{Height of the Building}/\text{Number of Floors}) < 1.15 * \text{Story Height}$$

M-RAM ARCHITECTURE AND IMPLEMENTATION

Getting information processes to share a common syntax is a major problem. Fig. 4 demonstrates how M-RAM deals with this problem, presenting the architecture that was applied in the prototype development. One of the most important characteristics of the proposed architecture is the possibility of geographical dispersion of the reasoning mechanisms. The idea was to allow the individual agents to be developed and maintained in different parts of the world using a ''plug-and-play'' architecture, leaving the agents where they were developed. This allows a faster client response that does not need to download all components and databases to its machine and allows easy agent maintenance and upgrade.

In the actual M-RAM implementation, a WWW interface written in Java (e.g., first tier) receives the expected data as input and launches the manager program (e.g., second tier). The manager program decides which server will perform which operation on the data, sending the required data to the location where the application is situated (e.g., third tier). Each application solves a part of the problem, sending back this partial solution to the manager program. The manager's task is to coordinate those partial solutions, providing a consistent solution for the problem as a whole. Finally, the manager program sends the solution or possible solutions of the structural design problem back to a WWW interface, which displays it to the user.

In this implementation, Java provides the mobile code foundation, and the Common Object Broker Architecture (CORBA) (Mowbray and Ruh 1997) provides the distributed object infrastructure. CORBA objects can be defined as binary components that remote clients can access via method invocations. Clients don't need to know where the distributed object resides, what operation a system executes on, or how the server object is implemented. What the client needs to know is the interface the server object publishes. This interface serves as a binding contract between client and servers (Orfali and Harkey 1997). CORBA-distributed object technology allows us to put together complex client/server information systems by simply assembling and extending components.

In the M-RAM architecture, CORBA's Object Request Broker (ORB) is the object bus. It lets objects transparently make requests to and receive responses from other objects, locally or remotely. It contains real-time information describing the function a server provides and its parameters. The ORB intercepts the call and is responsible for finding an object that can implement the request, pass it the parameters, invoke its methods, and return the results. The client just needs to know the object interface.

M-RAM enforces collaboration by writing code for the two sides of the collaboration. The next step would be the development of intelligent components that do more than just interoperate, but instead collaborate at the semantic level to get a job done. The trick, however, is to get components with no previous knowledge of each other to collaborate. To get to that point, we will need standards that set the rules of engagement for different component interaction boundaries. The Knowledge Query and Manipulation Language (KQML) (Finin et al. 1992), which is both a message format and a message-handling protocol to support run-time knowledge-sharing among agents, can provide these standards for a future implementation of an advanced M-RAM model.

ILLUSTRATIVE EXAMPLE

To test the feasibility of the proposed model, an M-RAM prototype was developed. To see how the prototype works and its design recommendations, consider, for example, the design of a hypothetical office building in Boston, Massachusetts. The specifications of the new design are for 85 floors, but because the building will be located in Boston's financial area near Logan Airport, the architect wants to keep the total height of the building as low as possible. To do so, he defined the total height to be around 280 m. The specifications also require that the structural material be steel because of New England's cold winters and labor characteristics, making the execution of massive concrete structures difficult and expensive. Finally, because of space requirements, the architect is also specifying 12 m of space between columns.

To be able to run M-RAM with these information requirements, the designer first needs to open the M-RAM project home page at http://cee-ta.mit.edu. This home page includes an open M-RAM interface button, clicking on which causes the M-RAM interface (Fig. 5) to pop onto the computer screen.

This easy-to-use interface allows the designer to interact with the three available agents. If the designer has any doubt about the input data, the available interface help can be used. Clicking the interface help menu automatically opens a browser with information about M-RAM attributes.

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

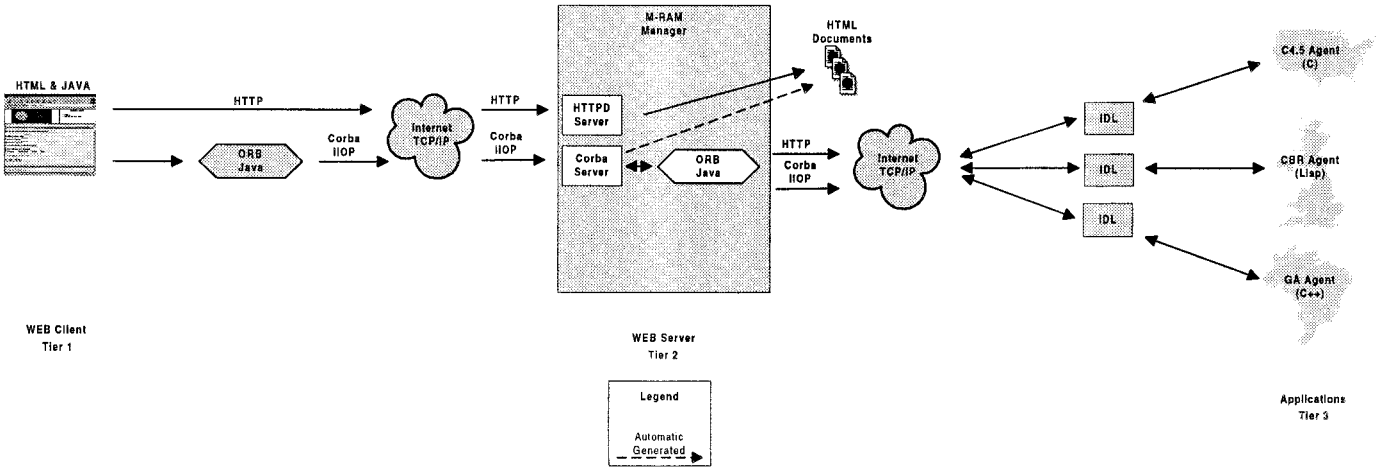


图4. M-RAM架构

在设计方案中，目标函数对那些使楼层数量、层高与建筑高度之间保持比例关系的方案予以加分。对于满足以下等式的所有基因组，目标函数均给予10分：

0.95 层高 < (建筑高度/ *

楼层数) < 1.15 层高 *

M-RAM架构与实现

使不同的信息处理过程共享统一的语法是一个重大难题。图4展示了M-RAM如何解决这一问题，并给出了在原型开发中所采用的架构。该架构的一个重要特征是推理机制能够实现地理上的分布式部署。其设计理念是，通过“即插即用”的架构，允许各个智能体在世界不同地区分别开发与维护，并将其保留在原开发地。这不仅能够加快客户端响应速度，使其无需将所有组件和数据库下载至本地，还便于智能体的维护与升级。

在实际的M-RAM实现中，一个用Java编写的WWW接口（例如第一层）接收预期的输入数据，并启动管理程序（例如第二层）。管理程序决定由哪台服务器对数据执行何种操作，并将所需数据发送至应用程序所在的节点（例如第三层）。每个应用程序负责求解问题的一部分，并将其局部解返回给管理程序。管理程序的任务是协调这些局部解，从而为整个问题提供一个一致的解决方案。最后，管理程序将结构设计问题的解或可能的解返回至WWW接口，并由该接口呈现给用户。

在该实现中，Java提供了移动代码的基础，而公共对象请求代理体系结构（CORBA）（Mowbray和Ruh，1997）则提供了分布式对象的基础设施。CORBA对象可被定义为二进制组件，远程客户端可通过方法调用来访问这些组件。客户端无需了解分布式对象的具体位置、系统执行的具体操作，以及服务器对象的实现方式。客户端只需知晓服务器对象所公开的接口。该接口充当客户端与服务器之间的绑定契约（Orfali和Harkey，1997）。借助CORBA分布式对象技术，我们只需通过组装和扩展组件，即可构建复杂的客户端/服务器信息系统。

在M-RAM架构中，CORBA的对象请求代理（ORB）充当对象总线。它使对象能够以透明的方式向本地或远程的其他对象发出请求并接收响应。ORB包含实时信息，用于描述服务器所提供的功能及其参数。ORB会拦截调用，并负责查找能够实现该请求的对象，将参数传递给该对象，调用其方法，并返回结果。客户端只需了解对象的接口即可。

M-RAM通过为协作的双方编写代码来促进协作。下一步将是开发智能组件，这些组件不仅能够实现互操作，还能够在语义层面进行协作，以完成特定任务。然而，其难点在于如何使彼此之间缺乏先验知识的组件实现协作。为此，我们需要制定一系列标准，以明确不同组件交互边界的行为规范。知识查询与操作语言（KQML）（Finin等，1992）既是一种消息格式，也是一种消息处理协议，用于支持代理之间的运行时知识共享，可为未来高级M-RAM模型的实现提供相应的标准依据。

示例

为验证所提出模型的可行性，开发了一款M-RAM原型系统。为了考察该原型系统的运行情况及其设计建议，以下以位于马萨诸塞州波士顿的一座假设办公建筑的设计为例进行说明。新设计方案的楼层数为85层，但由于该建筑将坐落于波士顿金融区、靠近洛根国际机场，建筑师希望尽可能降低建筑总高度。为此，他将建筑总高度定为约280米。此外，鉴于新英格兰地区冬季严寒以及当地劳动力特点，结构材料须采用钢材，因为采用大体积混凝土结构不仅施工难度大，而且成本高昂。最后，出于空间需求的考虑，建筑师还规定柱间净距为12米。

为了能够在满足这些信息需求的情况下运行M-RAM，设计人员首先需要访问位于http://cee-ta.mit.edu的M-RAM项目主页。该主页包含一个“打开M-RAM界面”按钮，单击该按钮后，M-RAM界面（图5）将弹出并显示在计算机屏幕上。

这一易于使用的界面使设计人员能够与三个可用的代理进行交互。若设计人员对输入数据存有疑问，可使用界面提供的帮助功能。单击界面帮助菜单后，浏览器将自动打开并显示有关M-RAM属性的相关信息。

J. Struct. Eng. 2000.126:733-742.

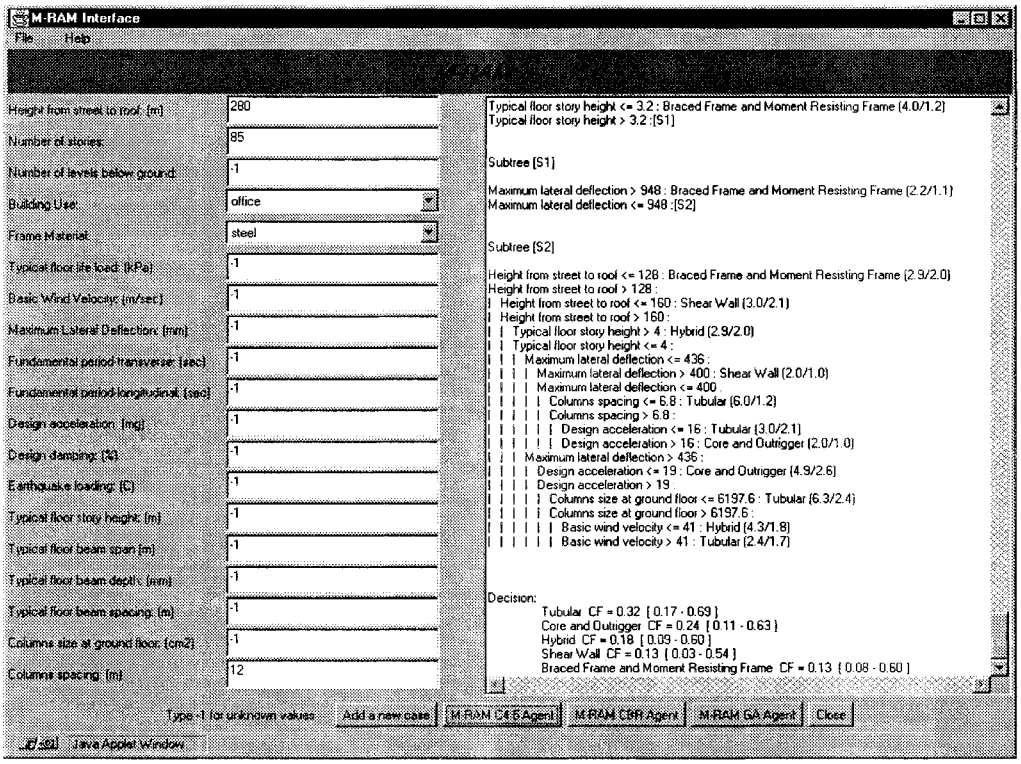


FIG. 5. M-RAM Interface with C4.5 Agent Output

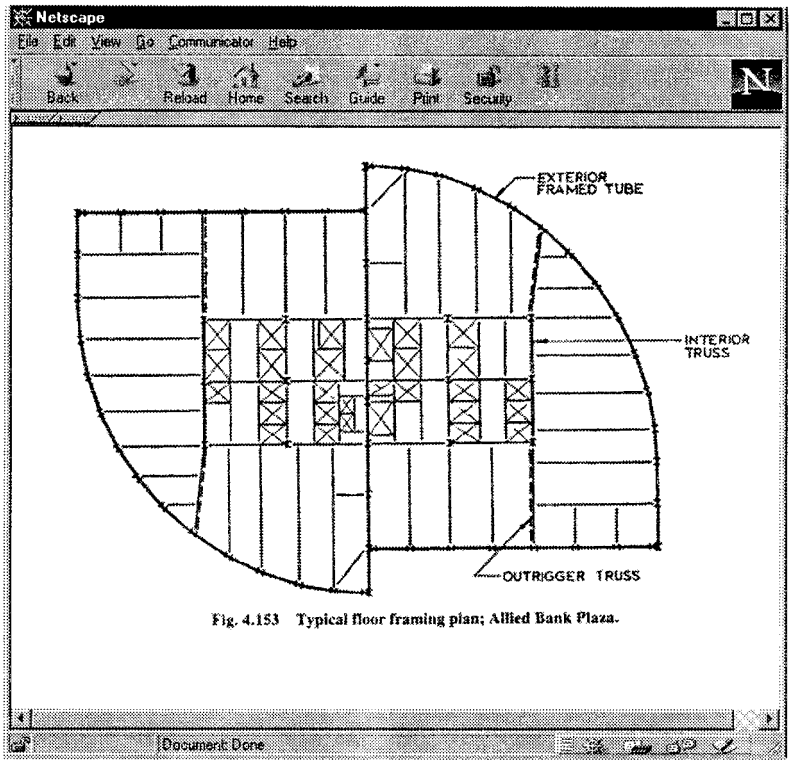


FIG. 6. Structural Detail of Example's Matching Case

To run the classification agent, the designer needs to click in the M-RAM C4.5 agent button after typing into the M-RAM interface the values for the design problem requirements (typing "−1" for unknown values). When the designer clicks the M-RAM C4.5 agent button, the values of the attributes are sent to the manager, which locates the classification agent and sends the information to the server where the C4.5 engine is located. The C4.5 engine classifies the design problem, sending the classification solution back to M-RAM manager, which displays it in the text area of the M-RAM interface, as shown in Fig. 5.

The output of the classification agent can be divided into

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

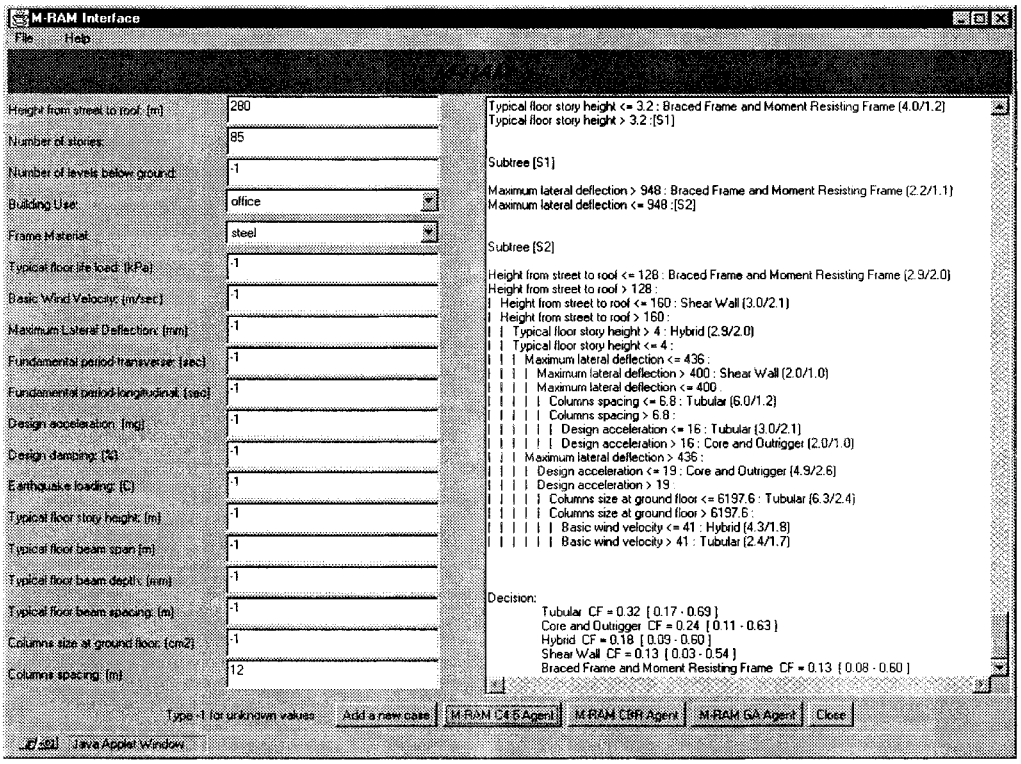


图5. 带有C4.5代理输出的M-RAM接口

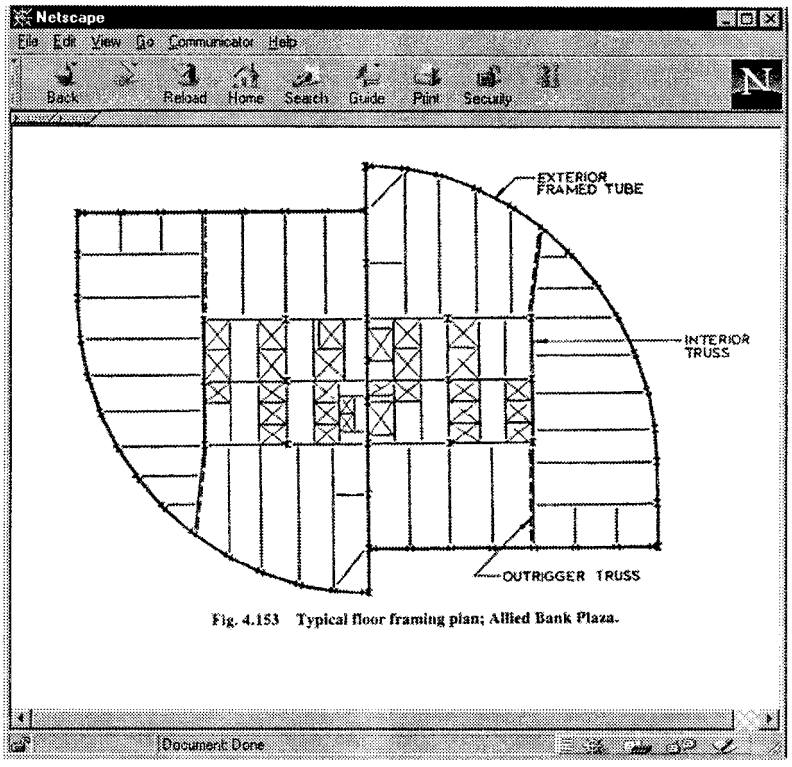


图6. 示例匹配案例的结构细节

要运行分类代理，设计人员需在 M-RAM 界面中输入设计问题的需求值（对于未知值输入“−1”）后，单击 M-RAM C4.5 代理按钮。当设计人员单击该按钮时，各属性的取值将被发送至管理器，由管理器定位并调用分类代理，并将该信息发送至部署有C4.5引擎的服务器。C4.5引擎对设计问题进行分类，并将分类结果返回给M-RAM管理器，由其在M-RAM界面的文本区域中显示，如图5所示。

分类代理的输出可分为

J. Struct. Eng. 2000.126:733-742.

three parts. The first part prints the information input by the designer, the second presents the actual decision tree, and the third presents the classification results. The example design (e.g., height from the street to the roof = 280 m; number of stories = 85; building use = office; frame material = steel; column spacing = 12 m; and all other attributes = −1) was classified as a tubular system with a probability somewhere between 0.17 and 0.69, the best guess being 0.32 [certainty factor (CF) is equal to 0.32]. CF is obtained during the testing stage of C4.5's training process, where the lower and upper bond and the confidence interval are defined for each class. M-RAM is thus informing the designer that the majority of buildings with characteristics similar to the one being designed were designed with tubular structural systems to resist lateral loads. With this information, M-RAM is ready to run its CBR agent to retrieve from its database of building designs the one that is the best fit, compared with the example design. To do so, the designer just needs to press the M-RAM CBR agent button. The manager then sends the design attributes added to the class defined by the C4.5 agent to the server where the CBR agent is located. The CBR agent retrieves from its memory the URL of the best matching case and sends this information back to the manager, which displays it to the user in the M-RAM interface text area. The CBR agent retrieved as a matching case for the design example a case instance called Allied Bank Plaza in Houston, Texas, linking the design solution to the retrieved case WWW URL. With this URL, the M-RAM manager launches a browser window that pops up in the designer computer with the home page of the retrieved case. From this home page, the user can go to a text description of the retrieved case, to a page with structural information on the case in tabular format, to a page with graphical structural details (Fig. 6), and to a page with an index of all cases of the same class as the one being designed. From this index, the designer is able to browse among available cases.

The retrieved case is a building with height from the street to the roof equal to 296 m, 71 floors, 4 levels below ground, office use, structural steel, 2.5 kPa of typical floor life load, 196 kN/m of basic wind velocity, H/500 maximum lateral deflection for 100 years return, 1% serviceability for design damping, story height equal to 4 m, 15.2 m of beam span, 0.53 m of beam depth, 4.6 m of beam spacing, column size of 1,016 by 610 mm in perimeter and 610 by 610 mm interior, and columns spacing of 4.6 m perimeter and 9.15 by 6.1 m interior. By comparing the problem requirements with the retrieved case attributes, it is possible to see that the retrieved case matches the specifications on the following attributes: height from the street to the roof, building use, and structural material. The retrieved case is a building approximately as tall as the one being designed, built with the same structural material and for the same use. Unfortunately, the retrieved case doesn't completely match the specifications for the other two requirements: number of floors and column spacing.

The building being designed has almost the same total height compared with the retrieved case, but because it has more floors, it is possible to conclude that it is going to have shorter floor-to-floor heights. The retrieved solution does not help the designer to deal with shorter floor-to-floor heights and larger distance among columns. The third M-RAM agent is then used to transform the retrieved solution by mixing parts of partial matches available in M-RAM's database, retrieved by the CBR agent.

To do so, the designer needs to press the M-RAM GA agent button. The manager then sends the design attributes that will be used as input to the objective function. The genetic algorithm agent runs for 1,500 generations, mixing bits and parts of the design cases provided by the CBR agent, and sends its solution back to the manager, which displays it to the user in the M-RAM interface text area.

The output of the genetic algorithm agent can be divided into two parts: The first prints statistical information about the genetic algorithm, and the second presents the genetic algorithm-proposed solution to the design problem. The M-RAM recommendation is a building with height from the street to the roof equal to 275 m, 82 floors, 4 levels below ground, office use, structural steel, 3.0 kPa of typical floor life load, 52 m/s of basic wind velocity, 522 mm maximum lateral deflection, fundamental period—transverse of 5 s, fundamental period—longitudinal of 6.5 s, design acceleration of 27 mg, 2% serviceability for design damping, story height equal to 3 m, 23 m of beam span, 1.63 m of beam depth, 4.9 m of beam spacing, column size of 70,728 cm^2, and column spacing of 12.15 m. It proposed a tall building with the same use, structural material, and nearly the same height, number of floors, and column spacing as that indicated in the problem specification. As expected, it has shorter floor-to-floor height, compared with the case retrieved by the CBR agent.

M-RAM classifies a design based on incomplete structural information. It retrieves a case from past experience, providing some insight into what occurred in the past so that previous mistakes can be avoided. Finally, it provides some recommendations for the values of the unknown structural parameters. M-RAM generates possible design alternatives, based on the specifications and available knowledge. This allows designers to evaluate different alternatives.

CONCLUSIONS

The use of the model and prototype presented in this paper improves the design process in time and quality. The required design-analysis cycle does not need to be repeated several times during the design process because M-RAM provides the designer with a better initial input for the design process. In addition, the ability of the system to retrieve lessons learned in previous, similar designs allows the designer to correct problems, avoiding repeating past mistakes in new designs. Usually the information at the conceptual design stage is not sufficient for an engineering analysis to determine the feasibility of a proposed solution. The M-RAM model supports designers by providing needed information in an organized and reliable structure.

This model is an improvement over the existing design process. Designers depend solely on their knowledge and experience to provide the assumptions they make during the initial stages of structural design, as well as the specifications for the design artifact. These initial assumptions have a strong impact on the quality of the final design and consequently on the quality of the final artifact. This model improves the conceptual design by supporting the designer's knowledge and experience. The well-organized database added by different functions provided by the model helps the designer to classify the design problem, retrieve useful past experience, and adapt past experience, providing the needed information to improve the quality and speed of the design process. In addition, the model provides a structural representation for providing information to the designer. Hence, the users will not be exposed to radical changes in their work technologies or be replaced by an automated computer designing system. This model supports the actual design process by providing the required information that helps the designer to elaborate on the design.

M-RAM can help not only the professional designers, but also students. It could be a very powerful tool to support structural engineering education, since M-RAM would be able to help students to quickly assimilate knowledge by learning with past design examples. Special tutors can easily be developed as an add-on to the proposed model.

In summarizing this work, the main contributions can be divided into two parts. First, a model for representing the con-

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

该界面分为三个部分：第一部分显示设计者输入的信息，第二部分展示实际的决策树，第三部分呈现分类结果。示例设计方案（例如：建筑高度自地面至屋面=为280 m；层数=为85层；建筑用途=为办公；框架材料=为钢；柱网间距=为12 m；其余属性= －均为1）被归类为筒体结构体系，其概率介于0.17至0.69之间，最佳估计值为0.32［置信度因子（CF）等于0.32］。置信度因子是在C4.5算法训练过程的测试阶段获得的，此时会为每一类别确定下限、上限及置信区间。因此，M-RAM系统向设计者提示：与当前设计特征相似的大多数建筑均采用筒体结构体系来抵抗侧向荷载。基于这一信息，M-RAM系统已准备好调用其案例推理（CBR）代理，从其建筑方案数据库中检索出与示例设计方案最为匹配的方案。为此，设计者只需点击M-RAM的CBR代理按钮。随后，管理模块将由C4.5代理所定义类别的设计属性发送至CBR代理所在的服务器。CBR代理从其知识库中检索出最匹配案例的URL，并将其反馈给管理模块，由管理模块在M-RAM界面的文本区域中向用户展示。经CBR代理检索，与该示例设计相匹配的案例实例为位于德克萨斯州休斯顿的Allied Bank Plaza，并将该设计方案与所检索案例的Web URL相关联。借助该URL，M-RAM管理模块将在设计者的计算机上弹出一个浏览器窗口，显示所检索案例的首页。通过该首页，用户可进一步访问该案例的文字说明、以表格形式呈现的结构信息页面、包含结构详图的图形页面（图6），以及与当前设计同属一类的所有案例索引页面。借助该索引，设计者可在现有案例中进行浏览。

所检索的案例为一栋建筑，其从街道至屋面的高度为296米，共71层，地下4层，用途为办公，采用钢结构，典型楼层活荷载为2.5 kPa，基本风速为196 kN/m，按100年重现期要求的最大侧向位移限值为H/500，设计阻尼比取1%，层高为4米，梁跨度为15.2米，梁截面高度为0.53米，梁间距为4.6米，柱网尺寸为：外围柱1,016 mm × 610 mm、内区柱610 mm × 610 mm，外围柱间距为4.6米，内区柱间距为9.15 m × 6.1 m。通过将本问题的需求与所检索案例的各项属性进行对比可知，该案例在以下属性上与设计要求相符：从街道至屋面的高度、建筑用途及结构材料。所检索案例的建筑高度与待设计建筑大致相当，且均采用相同的结构材料、用于相同的功能用途。然而，遗憾的是，该案例在另外两项关键指标——楼层数及柱网布置——上未能完全满足设计要求。

所设计的建筑与检索到的案例相比，总高度几乎相同，但由于其楼层数更多，因此可以推断其层高较短。然而，该检索到的解决方案无法帮助设计者应对层高较短且柱间距较大的情况。随后，第三位M-RAM智能体通过组合M-RAM数据库中由CBR智能体检索到的部分匹配方案，对检索到的解决方案进行改造。

为此，设计者需点击M-RAM GA代理按钮。随后，管理器将设计属性作为输入传递给目标函数。遗传算法代理运行1,500代，对CBR代理提供的设计实例中的位和部件进行重组，并将其求解结果反馈给管理器；管理器则在M-RAM界面的文本区域中向用户展示该解。

遗传算法代理的输出可分为两部分：第一部分输出有关遗传算法的统计信息，第二部分则给出遗传算法针对该设计问题所提出的解决方案。M-RAM推荐方案为一栋建筑，其从地面至屋面的高度为275米，共82层，地下4层，用途为办公，采用钢结构，标准楼层活荷载为3.0千帕，基本风速为52米/秒，最大侧向位移为522毫米，横向自振周期为5秒，纵向自振周期为6.5秒，设计加速度为27毫克，设计阻尼比为2%，层高为3米，梁跨度为23米，梁截面高度为1.63米，梁间距为4.9米，柱截面面积为70,728平方厘米，柱间距为12.15米。该方案提出了一栋与问题说明中所给定条件在用途、结构材料以及高度、层数和柱间距方面几乎一致的高层建筑。正如预期，与CBR代理检索到的案例相比，其层间高度更小。

M-RAM根据不完整的结构信息对设计方案进行分类。它从以往的经验中检索相关案例，从而为过去发生的情况提供一定参考，以避免重蹈覆辙。最后，它针对未知的结构参数值给出若干建议。M-RAM还基于设计规范和现有知识生成若干可行的设计方案，使设计人员能够对不同方案进行评估。

结论

本文所提出的模型与原型的应用，在时间和质量两方面均提升了设计流程的效率。由于M-RAM能够为设计过程提供更为优质的初始输入，设计过程中无需反复进行多次设计—分析循环。此外，该系统具备检索以往类似设计中经验教训的能力，使设计人员得以及时纠正问题，避免在新设计中重蹈覆辙。通常，在概念设计阶段获取的信息尚不足以支撑工程分析以判定所提方案的可行性；而M-RAM模型则通过以结构化且可靠的方式提供所需信息，为设计人员提供了有力支持。

该模型是对现有设计流程的改进。在结构设计的初始阶段，设计师仅依赖自身的知识与经验来做出假设，并确定设计对象的规格要求。这些初始假设对最终设计的质量乃至最终成果的质量具有重要影响。本模型通过有效支撑设计师的知识与经验，提升了概念设计的水平。模型所集成的、由各项功能共同构建的结构化数据库，有助于设计师对设计问题进行分类、检索并借鉴以往的有用经验，从而为提升设计过程的质量与效率提供所需信息。此外，该模型还以结构化的形式向设计师传递相关信息。因此，用户无需面对工作方式的根本性变革，也无需被自动化的计算机设计系统所取代。本模型通过提供必要的信息，切实支持实际的设计过程，帮助设计师进一步完善设计方案。

M-RAM不仅能够为专业设计人员提供帮助，也能惠及学生。作为一种强有力的工具，它有望支持结构工程教育，因为M-RAM能够通过结合过往的设计实例，帮助学生快速掌握相关知识。此外，还可基于所提出的模型便捷地开发专门的辅导模块。

综上所述，本工作的主要贡献可分为两个部分。首先，提出了一种用于表示……的模型。

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by Tsinghua University on 03/27/17. Copyright ASCE. For personal use only; all rights reserved.

ceptual phase of structural design was developed. This model provides initial building blocks for representing design knowledge in terms of the reasoning process used by designers in generating an artifact satisfying their design intents. This model also takes into consideration the different reasoning mechanisms that collaborate to solve the design problem and is used to provide active computer support for designers in the reasoning process.

The second contribution was the development of a distributed open-system architecture that integrates different reasoning mechanisms. This novel system architecture was applied where different reasoning mechanisms can be located in different servers dispersed in diverse locations around the world. This research had the objective of integrating agents that were not intended to collaborate to reach a common solution. It provided a solution that achieves a given goal with the integration of different reasoning mechanisms, providing a solution to a problem by using the power of cooperation. It also provided a framework in which collaboration could occur while reducing overhead of maintenance, limitation of scalability, or danger of obsolescence in such a rapidly changing field. And finally, it uses a combination of graphs, text, and tables to convey the information, with the easy possibility of adding simulation capabilities without a major change in the proposed system architecture.

APPENDIX. REFERENCES

Bailey, S. F., and Smith, I. F. C. (1994). ''Case-based preliminary building design.'' *J. Computing in Civ. Engrg.*, ASCE, 8(4), 454–468.

''Creating a case-base using CASL.'' (1996a). University of Wales—Aberystwith, U.K.

Fenves, S. J., Rivard, H., Gomez, N., and Chiou, S. C. (1995). ''Conceptual structural design in seed.'' *J. Arch. Engrg.*, ASCE, 1(4), 179–186.

Finin, T., Fritzson, R., and McKay, D. (1992). ''A language and protocol to support intelligent agent interoperability.'' *Proc., CE&CALS Conf.*, Society for Computer-Aided Engineering, Rockford, Ill.

Finin, T., Fritzson, R., McKay, D., and McEntire, R. (1994). ''KQML as an agent communication language.'' *Proc., 3rd Int. Conf. on Information and Knowledge Mgmt.*, ACM Press, New York.

Gasser, L., and Hill, R. W. (1990). ''Coordinated problem solver.'' *Annual review of computer science*, Vol. 4, J. F. Traub, B. J. Grosz, B. W. Lampson, and N. J. Nilsson, Annual Reviews Inc., Palo Alto, Calif., 203–253.

An introductory guide to Caspian. (1996b). University of Wales—Aberystwith, U.K.

Kiliccote, H., and Garrett Jr., J. H. (1998). ''Agent-based support for building code usage in building design systems.'' *Proc., Struct. Engrs. World Congr.*, Elsevier Science, New York.

Kolodner, J. (1993). *Case-based reasoning*, Morgan Kaufmann, San Mateo, Calif.

Kowalczyk, R. M., Sinn, R., and Kilmister, M. B. (1995). *Structural systems for tall buildings*, Council on Tall Buildings and Urban Habitat —McGraw-Hill, New York.

Koza, J. R. (1996). *Genetic programming—On the programming of computers by means of natural selection*, MIT Press, Cambridge, Mass.

Lander, S. E., and Lesser, V. R. (1992). ''Customizing distributed search among agents with heterogeneous knowledge.'' *Proc., 1st Int. Conf. on Information and Knowledge Mgmt.*, Y. Yesha, ed., Calgary, Canada. 335–344.

Leake, D. B. (ed.). (1996). ''CBR in context: The present and the future.'' *Case-based reasoning—Experience, lessons, and future directions*, AAAI Press/MIT Press, Cambridge, Mass.

Maes, P. (1991). *Designing autonomous agents: Theory and practice from biology to engineering and back*, MIT Press, Cambridge, Mass.

Maher, M. L. (1987). ''Expert systems for structural design,'' *J. Computing in Civ. Engrg.*, ASCE, 1(4), 270–283.

Maher, M. L., and de Silva Garza, A. G. (1996). ''The adaptation of structural systems designs using genetic algorithms.'' *Information processing in civil and structural engineering design*, B. Kumar, ed., Civil-Comp Press, Edinburgh, U.K.

Maher, M. L., and Fenves, S. J. (1985). ''HI-RISE: An expert system for the preliminary structure design of high rise buildings.'' *Knowledge engineering in computer-aided design*, J. S. Gero, ed., North-Holland, Amsterdam.

Manfaat, D., Duffy, A. H. B., and Lee, B. S. (1997). *SPIDA for the utilization of layout design experience*, University of Strathclyde, Scotland.

Minsky, M. (1988). *The society of mind*, Touchstone/Simon and Schuster, New York.

Mowbray, T. J., and Ruh, W. A. (1997). *Inside Corba—Distributed object standards and applications*, Addison-Wesley Longman, Inc., Reading, Mass.

Orfali, R., and Harkey, D. (1997). *Client/server programming with Java and Corba*, Wiley, New York.

Peña-Mora, F. (1994). ''Design rationale for computer supported conflict mitigation during the design-construction process of large-scale civil engineering systems.'' PhD thesis, Massachusetts Institute of Technology, Cambridge, Mass.

Peña-Mora, F., Sriram, D., and Logcher, R. (1995). ''Design rationale for computer-supported conflict mitigation.'' *J. Comp. in Civ. Engrg.*, ASCE, 9(1), 57–72.

Pouangare, C. C. (1986). ''Strategies for the conceptual design of structures.'' MS thesis, Massachusetts Institute of Technology, Cambridge, Mass.

Quinlan, J. R. (1993). *C.4.5 programs for machine learning*, Morgan Kaufmann, San Mateo, Calif.

Rational Software Corporation. (1997). ''Unified modeling language—Notation guide, UML summary, process-specific extensions, and UML semantics.'' ⟨*http://www.rational.com*⟩.

Soibelman, L. (1998). ''The exploration of an integrated representation for the conceptual design phase of structural design for tall buildings through distributed multi-reasoning algorithms.'' PhD thesis, Massachusetts Institute of Technology, Cambridge, Mass.

Talukdar, P. S., DeSouza, R., Quadrel, R., and Ramesh, V. C. (1992). ''A-teams: Multi-agent organizations for distributed iteration.'' *Tech. Rep. EDRC-18-30-92*, Engrg. Des. Res. Ctr., Carnegie Mellon University, Pittsburgh, Pa.

Wall, M. (1996). *GAlib: A C++ library of genetic algorithm components, Version 2.4, Revision B.* Mech. Engr. Dept., Massachusetts Institute of Technology, Cambridge, Mass.

Downloaded from ascelibrary.org by GEORGE MASON UNIVERSITY on 07/27/13. Copyright ASCE. For personal use only; all rights reserved.

J. Struct. Eng. 2000.126:733-742.

结构设计的概念阶段已被开发出来。该模型为以设计师在生成满足其设计意图的构件时所采用的推理过程来表征设计知识提供了初始构建模块。此外，该模型还考虑了协同解决设计问题的不同推理机制，并用于在推理过程中为设计师提供主动的计算机辅助支持。

第二项贡献是开发了一种分布式开放系统架构，该架构能够集成不同的推理机制。这一新颖的系统架构允许不同的推理机制部署在全球各地的多台服务器上。本研究旨在整合那些原本并未设计用于协作以达成共同解决方案的智能体，通过集成多种推理机制来实现既定目标，从而借助协作的力量解决实际问题。此外，该研究还提供了一个框架，能够在降低维护开销、克服可扩展性瓶颈并规避在这一快速变化领域中技术过时风险的同时，促进协作的开展。最后，该系统采用图形、文本与表格相结合的方式呈现信息，并且在不对所提出的系统架构进行重大改动的情况下，即可轻松添加仿真功能。

附录. 参考文献

Bailey, S. F. 和 Smith, I. F. C.（1994）.“基于案例的初步建筑设计”。*J. Computing in Civ. Engrg.*，ASCE，8(4)，454–468页。“利用CASL构建案例库”。（1996a）。英国威尔士大学—阿伯里斯特威斯分校。Fenves, S. J.、Rivard, H.、Gomez, N. 和 Chiou, S. C.（1995）。“Seed中的概念化结构设计”。*J. Arch. Engrg.*，ASCE，1(4)，179–186页。Finin, T.、Fritzson, R. 和 McKay, D.（1992）。“一种用于支持智能代理互操作性的语言与协议”。*Proc., CE&CALS Conf.*，计算机辅助工程学会，伊利诺伊州罗克福德市。Finin, T.、Fritzson, R.、McKay, D. 和 McEntire, R.（1994）：“KQML作为代理通信语言”。*Proc., 3rd Int. Conf. on Informa-tion and Knowledge Mgmt.*，ACM出版社，纽约。Gasser, L. 和 Hill, R. W.（1990）：“协同式问题求解器”。*Annualreview of computer science*，第4卷，J. F. Traub、B. J. Grosz、B. W. Lampson 和 N. J. Nilsson 主编，Annual Reviews Inc.，加利福尼亚州帕洛阿尔托，203–253页。*An introductory guide to Caspian*。（1996b）。英国威尔士大学—阿伯里斯特威斯分校。Kiliccote, H. 和 Garrett Jr., J. H.（1998）：“基于代理的支持——在建筑设计系统中应用建筑规范”。*Proc., Struct. Engrs.World Congr.*，爱思唯尔科学出版社，纽约。Kolodner, J.（1993）：*Case-based reasoning*，摩根考夫曼出版社，加利福尼亚州圣马特奥。Kowalczyk, R. M.、Sinn, R. 和 Kilmister, M. B.（1995）：*Structural systems for tall buildings*，高层建筑与城市栖息地理事会——麦格劳-希尔出版社，纽约。Koza, J. R.（1996）。*Genetic programming—On the programming of computers by means of natural selection*，麻省理工学院出版社，马萨诸塞州剑桥市。Lander, S. E. 和 Lesser, V. R.（1992）。“在具有异质性知识的智能体之间定制分布式搜索”。*Proc., 1st Int. Conf. on Information and Knowledge Mgmt.*，Y. Yesha 主编，加拿大卡尔加里。335–344页。Leake, D. B.（编）（1996）。“案例推理的背景：现状与未来”。*Case-based reasoning—Experience, lessons, and future directions*，美国人工智能协会出版社/麻省理工学院出版社，马萨诸塞州剑桥市。Maes, P.（1991）。*Designing autonomous agents: Theory and practice from biology to engineering and back*，麻省理工学院出版社，马萨诸塞州剑桥市。Maher, M. L.（1987）。“用于结构设计的专家系统”，*J. Computing in Civ. Engrg.*，美国土木工程师学会，1(4)，270–283页。Maher, M. L. 和 de Silva Garza, A. G.（1996）。“利用遗传算法对结构体系设计进行适应性调整”。*Information processing in civil and structural engineering design*，B. Kumar 主编，Civil-Comp 出版社，英国爱丁堡。Maher, M. L. 和 Fenves, S. J.（1985）。“HI-RISE：用于高层建筑初步结构设计的专家系统”。*Knowledge engineering in computer-aided design*，J. S. Gero 主编，荷兰阿姆斯特丹北荷兰出版社。Manfaat, D.、Duffy, A. H. B. 和 Lee, B. S.（1997）。*SPIDA for the utilization of layout design experience*，苏格兰斯特拉斯克莱德大学。Minsky, M.（1988）。*The society of mind*，Touchstone/Simon and Schuster 出版社，纽约。Mowbray, T. J. 和 Ruh, W. A.（1997）。*Inside Corba—Distributed object standards and applications*，Addison-Wesley Longman 公司，马萨诸塞州雷丁。Orfali, R. 和 Harkey, D.（1997）。*Client/server programming with Java and Corba*，威利出版社，纽约。F.（1994）。“大型土木工程系统设计—施工过程中计算机支持的冲突缓解的设计 rationale”。麻省理工学院博士学位论文，马萨诸塞州剑桥市。F.、Sriram, D. 和 Logcher, R.（1995）。“计算机支持的冲突缓解的设计 rationale”。*J. Comp. in Civ. Engrg.*，美国土木工程师学会，9(1)，57–72页。Pouangare, C. C.（1986）。“结构概念设计策略”。麻省理工学院硕士学位论文，马萨诸塞州剑桥市。Quinlan, J. R.（1993）。*C.4.5 programs for machine learning*，摩根·考夫曼出版社，加利福尼亚州圣马特奥市。Rational Software Corporation（1997）。“统一建模语言——符号指南、UML 概要、过程特定扩展及 UML 语义”。〈*http://www.rational.com*〉。Soibelman, L.（1998）。“通过分布式多推理算法探索高层建筑结构设计概念阶段的集成表示方法”。麻省理工学院博士学位论文，马萨诸塞州剑桥市。Talukdar, P. S.、DeSouza, R.、Quadrel, R. 和 Ramesh, V. C.（1992）。“A-teams：用于分布式迭代的多智能体组织”。*Tech. Rep. EDRC-18-30-92*，卡内基梅隆大学工程设计研究中心，宾夕法尼亚州匹兹堡市。Wall, M.（1996）。*GAlib: A C++ library of genetic algorithm components, Version 2.4, Revision B.*，麻省理工学院机械工程系，马萨诸塞州剑桥市。

J. Struct. Eng. 2000.126:733-742.

Downloaded from ascelibrary.org by Universidade do Porto FEUP on 03/27/20. Copyright ASCE. For personal use only; all rights reserved.